

Módulo 3 — Context Engineering

Capítulo 06 — RAG (Retrieval-Augmented Generation) como componente del Context Engineering

Sección 01 — Introducción a RAG como componente del Context Engineering

"Un modelo que no puede acceder a información relevante produce respuestas relevantes solo por azar. RAG convierte ese azar en arquitectura."

Objetivos de aprendizaje

- Comprender qué es RAG y por qué emerge como solución a un problema de diseño específico.
- Ubicar RAG dentro del marco más amplio del Context Engineering.
- Diferenciar RAG de otras estrategias para enriquecer el contexto: memoria, resumen e instrucciones del sistema.
- Establecer el mapa conceptual del capítulo antes de desarrollar cada componente.

El problema que RAG resuelve

En los capítulos anteriores construimos una arquitectura de contexto que incluye instrucciones del sistema, memoria conversacional y estrategias de compresión. Ese contexto es poderoso, pero tiene un límite que ninguno de esos mecanismos puede resolver por sí solo: el modelo no sabe lo que no vio durante su entrenamiento.

Un modelo de lenguaje de gran escala aprende durante el preentrenamiento. Después de ese proceso, su conocimiento queda congelado. No importa cuántas horas de cómputo se invirtieron ni cuántos parámetros tiene: si el evento que el usuario pregunta ocurrió después del corte de entrenamiento, si el documento que necesita es interno a la organización, o si la regulación que debe aplicar es específica del sector, el modelo simplemente no tiene esa información.

Frente a esta limitación, hay dos respuestas posibles.

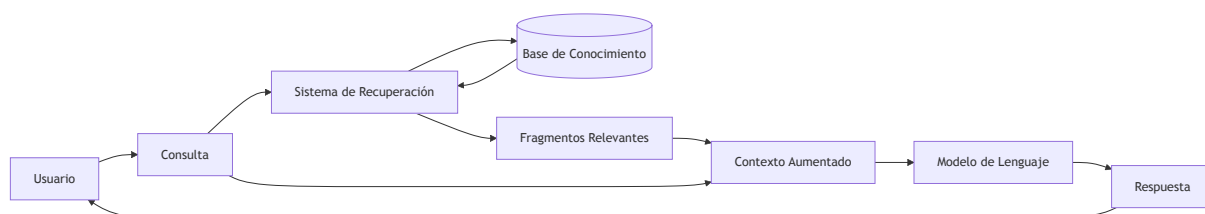
La primera consiste en actualizar el modelo. Reentrenarlo, ajustarlo con nuevos datos, mantenerlo sincronizado con el mundo que cambia. Esta respuesta es costosa, lenta y difícilmente sostenible para información que se actualiza con frecuencia.

La segunda consiste en diseñar el sistema para que, en el momento en que el usuario hace una consulta, el sistema recupere la información relevante y la entregue al modelo dentro del contexto. El modelo no necesita saberlo de antemano: lo aprende en el momento, en el contexto de cada consulta. Esta es la lógica de RAG.

Qué es RAG

Retrieval-Augmented Generation es una arquitectura de sistema que combina dos operaciones que, sin RAG, permanecen separadas: la recuperación de información relevante de un repositorio externo y la generación de una respuesta por parte de un modelo de lenguaje.

El nombre lo describe con precisión: la generación (Generation) se aumenta (Augmented) mediante recuperación (Retrieval). El modelo no genera desde su conocimiento interno solamente. Genera desde un contexto que el sistema construyó dinámicamente, seleccionando los fragmentos de información más relevantes para la consulta actual.



Este diagrama captura la idea central: la consulta del usuario no va directamente al modelo. Primero pasa por un sistema de recuperación que extrae los fragmentos más relevantes de una base de conocimiento. Esos fragmentos, junto con la consulta original, forman el contexto que el modelo recibe. El modelo genera su respuesta basándose en ese contexto aumentado.

RAG como componente del Context Engineering

En el capítulo 01 de este módulo definimos el Context Engineering como la disciplina que se ocupa de diseñar, construir y gestionar el contexto que un sistema de IA recibe en cada inferencia. Todo lo que se entrega al modelo en el prompt es contexto. El Context Engineering es la ingeniería de ese contenido.

Desde esa perspectiva, RAG es una estrategia de Context Engineering. Su función específica es responder una pregunta de diseño concreta: ¿cómo incorporo al contexto información externa que el modelo no tiene en su conocimiento interno?

Las otras estrategias que exploramos en este módulo responden preguntas diferentes:

Estrategia	Pregunta que responde
Instrucciones del sistema	¿Cómo defino el comportamiento, el rol y las restricciones del modelo?
Memoria conversacional	¿Cómo mantengo continuidad entre turnos y sesiones?
Resumen y compresión	¿Cómo gestiono el contexto cuando el historial supera la ventana?
RAG	¿Cómo incorporo conocimiento externo relevante para la consulta actual?

No son estrategias competidoras. Son complementarias. En una aplicación empresarial bien diseñada, las cuatro conviven y se coordinan dentro de la misma arquitectura.

Por qué RAG cambió el diseño de aplicaciones de IA

Antes de RAG, la forma dominante de especializar un modelo era el fine-tuning: ajustar los parámetros del modelo con datos del dominio específico. El fine-tuning produce un modelo

que "sabe" más sobre un tema, pero ese conocimiento sigue siendo estático. Si los datos del dominio cambian, el modelo no sabe que cambiaron.

RAG separó el conocimiento del modelo de la actualización del conocimiento. El modelo permanece genérico y estable. El conocimiento específico vive en un índice externo que puede actualizarse de forma independiente, sin necesidad de tocar el modelo. Esta separación tiene consecuencias prácticas profundas:

- Actualizar la información disponible al sistema requiere actualizar el índice, no reentrenar el modelo.
- El mismo modelo puede servir a múltiples dominios con distintos índices.
- La trazabilidad de las respuestas mejora: se puede identificar qué fragmento de qué documento respaldó cada respuesta.
- Los errores de información se pueden corregir editando el índice, no ajustando parámetros.

Qué contiene este capítulo

Este capítulo cubre el ciclo completo de RAG desde la perspectiva de un AI Engineer que debe diseñar, implementar y operar sistemas de recuperación en contextos reales.

Las secciones siguen esta progresión:

1. Limitaciones del conocimiento interno del modelo — Por qué RAG existe.
 2. Arquitectura de un sistema RAG — Cómo está compuesto.
 3. Embeddings y representación semántica — Cómo se representa el significado.
 4. Bases vectoriales y búsqueda por similitud — Dónde vive ese significado.
 5. Estrategias de recuperación — Cómo se seleccionan los fragmentos relevantes.
 6. Re-ranking y selección de contexto — Cómo se mejora la selección inicial.
 7. Diseño de un pipeline RAG empresarial — Cómo se ensambla todo en producción.
 8. Patrones y anti-patrones — Qué funciona y qué rompe sistemas reales.
 9. Caso de estudio completo — Una implementación de principio a fin.
 10. Laboratorio, checklist, resumen y autoevaluación.
-

Ideas clave

- RAG es una estrategia del Context Engineering que incorpora conocimiento externo al contexto de cada inferencia.
 - Separa el conocimiento del modelo del conocimiento del sistema, permitiendo actualizar uno sin tocar el otro.
 - No es un sustituto del fine-tuning: es una alternativa arquitectónica con ventajas e inconvenientes distintos.
 - En una arquitectura empresarial completa, RAG coexiste con memoria, instrucciones del sistema y estrategias de compresión.
-

Transición hacia la siguiente sección

Antes de profundizar en cómo RAG funciona, necesitamos entender con precisión por qué existe. La siguiente sección examina las limitaciones del conocimiento interno del modelo: el problema del corte de entrenamiento, el costo de actualizarlo y las restricciones que el conocimiento estático impone sobre aplicaciones que operan en dominios cambiantes.

"Un arquitecto no memoriza respuestas. Comprende problemas para poder diseñar soluciones."

Módulo 3 — Context Engineering

Capítulo 06 — RAG (Retrieval-Augmented Generation) como componente del Context Engineering

Sección 02 — Limitaciones del conocimiento interno del modelo

"El modelo sabe mucho. Pero lo que sabe dejó de ser verdad en algún momento entre su entrenamiento y tu consulta."

Objetivos de aprendizaje

- Comprender la naturaleza del conocimiento interno de un modelo de lenguaje y sus límites inherentes.
- Analizar el problema del corte de entrenamiento y sus consecuencias en aplicaciones reales.
- Evaluar el costo económico y operativo de actualizar el conocimiento vía reentrenamiento frente a la alternativa de actualizar un índice externo.
- Identificar los tipos de conocimiento que RAG cubre mejor que cualquier otra estrategia.

El conocimiento como instantánea

Un modelo de lenguaje de gran escala aprende durante el preentrenamiento. Ese proceso consume meses y recursos computacionales significativos. Su resultado es un modelo con miles de millones de parámetros que codifican, de manera distribuida e implícita, patrones extraídos de una fracción enorme del texto disponible hasta cierta fecha.

Ese "hasta cierta fecha" es el corte de entrenamiento, también llamado knowledge cutoff. A partir de ese momento, el conocimiento del modelo queda congelado. No porque el modelo haya dejado de funcionar, sino porque el proceso que lo habría actualizado —el preentrenamiento con nuevos datos— no volvió a ejecutarse.

Para comprender la dimensión del problema, consideremos lo que sucede en doce meses cualquiera en un dominio activo: nuevas regulaciones entran en vigor, organizaciones cambian de nombre o de estructura, tecnologías emerge o quedan obsoletas, pronunciamientos judiciales reinterpretan marcos legales, precios cambian, líderes de área se reemplazan, productos se descontinúan. Un modelo entrenado antes de esos cambios no sabe que ocurrieron. Si se le pregunta sobre ellos, puede alucinarse una respuesta plausible basada en lo que sí conoce.

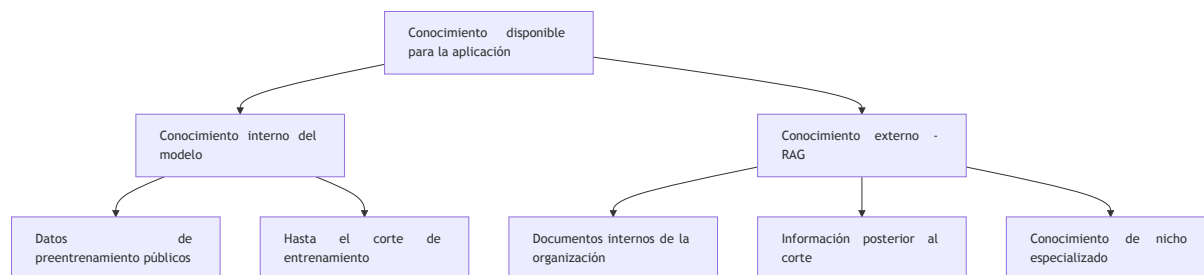
Las tres clases de conocimiento que el modelo no tiene

Para diseñar sistemas de recuperación, conviene distinguir tres clases de conocimiento que ningún modelo de lenguaje puede tener en su estado base:

Conocimiento temporal: Cualquier información generada después del corte de entrenamiento. Esto incluye desde noticias recientes hasta actualizaciones de versiones de software, pasando por cambios regulatorios o resultados de investigaciones publicadas en el último año.

Conocimiento privado: Documentos internos de la organización —políticas, procedimientos, contratos, manuales técnicos, registros de clientes— que por definición no formaron parte de los datos de preentrenamiento porque no son públicos. Un modelo nunca puede conocer el organigrama de una empresa, las tarifas internas de un contrato bilateral o los estándares específicos de un sistema propietario.

Conocimiento de nicho: Información altamente especializada que sí es pública pero que está subrepresentada en los datos de preentrenamiento. Normativas sectoriales obscuras, literatura técnica en idiomas de menor difusión, estándares industriales publicados por organismos de baja visibilidad. El modelo puede tener nociones sobre el tema, pero no la profundidad que una aplicación especializada requiere.



El costo de actualizar el modelo

La respuesta intuitiva al problema del conocimiento desactualizado es: "actualizar el modelo". En la práctica, esta respuesta tiene un costo que la hace impracticable para la mayoría de las aplicaciones.

El preentrenamiento de un modelo de lenguaje de gran escala requiere clústeres de GPUs de alta memoria durante semanas o meses, con costos que en la escala de los modelos modernos pueden superar el millón de dólares. Los equipos de ingeniería que desarrollan modelos propietarios —GPT, Claude, Gemini, Llama— tienen ciclos de actualización que se miden en meses, no en días.

El fine-tuning es una alternativa más liviana: en lugar de reentrenar desde cero, se ajustan los pesos del modelo con un conjunto de datos más pequeño y específico. Pero el fine-tuning también tiene limitaciones frente al problema del conocimiento dinámico:

- El modelo ajustado sabe lo que aprendió durante el fine-tuning. Cuando esa información cambia, el proceso debe repetirse.
- El fine-tuning no produce trazabilidad: el modelo no puede señalar qué dato específico respaldó una afirmación concreta.
- Para datos que se actualizan con frecuencia —normativas que cambian cada mes, tarifas que cambian cada semana, inventario que cambia en tiempo real— el ciclo de fine-tuning es insostenible.

Dimensión	Reentrenamiento	Fine-tuning	RAG
Costo inicial	Muy alto	Moderado	Bajo-Moderado
Latencia de actualización	Semanas-Meses	Días-Semanas	Minutos-Horas
Trazabilidad de respuestas	Nula	Nula	Alta
Adecuado para datos muy dinámicos	No	No	Sí
Adecuado para comportamiento general	Sí	Sí	No
Riesgo de catastrophic forgetting	Alto	Moderado	Nulo

El concepto de *catastrophic forgetting* merece una nota específica. Cuando un modelo se ajusta con nuevos datos, puede perder parte del conocimiento que tenía antes. Este fenómeno, conocido como olvido catastrófico, es una limitación estructural de los modelos basados en redes neuronales. Al actualizar un índice de RAG, no existe este riesgo: el modelo permanece intacto.

El índice como alternativa al reentrenamiento

La lógica de RAG parte de una observación simple: si el conocimiento externo puede organizarse en un repositorio recuperable, no necesita estar en los parámetros del modelo. El modelo puede funcionar como un razonador genérico que opera sobre el conocimiento que el sistema le entrega en cada consulta.

Esta separación tiene consecuencias arquitectónicas relevantes:

Actualización desacoplada. El índice puede actualizarse en producción sin detener el modelo. Un cambio en la normativa tributaria puede reflejarse en el sistema en el tiempo que tarda en indexarse el nuevo documento, no en el tiempo que tarda en ajustarse el modelo.

Trazabilidad de las fuentes. Cuando el modelo genera una respuesta basada en contexto recuperado, el sistema puede registrar qué fragmentos de qué documentos alimentaron esa respuesta. En dominios regulados —finanzas, salud, derecho— esta trazabilidad puede ser un requisito no negociable.

Separación de dominios. El mismo modelo base puede servir a múltiples dominios con distintos índices. Un modelo genérico conectado a un índice de documentos legales se

comporta como un asistente legal. Conectado a un índice de manuales técnicos, como un asistente de soporte. El modelo no cambia; el conocimiento disponible sí.

Cuándo el modelo sí es suficiente

RAG no es siempre la solución correcta. Hay consultas que el modelo responde perfectamente desde su conocimiento interno: preguntas sobre conceptos generales, razonamiento lógico, redacción, traducción, análisis de texto proporcionado por el usuario. Para estas tareas, agregar un sistema de recuperación introduce latencia y complejidad sin beneficio.

La pregunta de diseño correcta es: ¿la respuesta a esta consulta depende de información que el modelo podría no tener o que podría estar desactualizada? Si la respuesta es sí, RAG es candidato. Si la respuesta es no, no es necesario.

Nota del Arquitecto

El error más común al decidir si usar RAG no es técnico: es estratégico. Los equipos tienden a indexar todo lo que tienen disponible y a conectarlo al modelo sin evaluar si esa información mejora o degrada las respuestas. Un índice con documentos obsoletos, mal segmentados o irrelevantes puede ser peor que no tener RAG: introduce ruido en el contexto y guía al modelo hacia afirmaciones incorrectas con aparente respaldo documental. La calidad del índice importa más que su tamaño.

Ideas clave

- El conocimiento interno del modelo es estático: queda congelado en la fecha de corte del entrenamiento.
- Hay tres clases de conocimiento que el modelo no puede tener: temporal, privado y de nicho.
- Actualizar el conocimiento vía reentrenamiento o fine-tuning es costoso, lento y no produce trazabilidad.

- RAG desacopla el conocimiento del sistema del conocimiento del modelo, permitiendo actualizar uno sin modificar el otro.
 - RAG no es siempre la solución correcta: conviene cuando la respuesta depende de información externa o desactualizada.
-

Transición hacia la siguiente sección

Ahora que entendemos el problema que RAG resuelve, podemos estudiar cómo lo resuelve. La siguiente sección introduce la arquitectura completa de un sistema RAG: sus fases, sus componentes y el flujo de información que los conecta.

"Un arquitecto no memoriza respuestas. Comprende problemas para poder diseñar soluciones."

Módulo 3 — Context Engineering

Capítulo 06 — RAG (Retrieval-Augmented Generation) como componente del Context Engineering

Sección 03 — Arquitectura de un sistema RAG

"RAG no es una función. Es un pipeline con dos fases distintas, cada una con sus propias decisiones de diseño."

Objetivos de aprendizaje

- Comprender la arquitectura completa de un sistema RAG como composición de componentes con responsabilidades diferenciadas.
- Distinguir la fase offline (indexación) de la fase online (recuperación e inferencia).
- Identificar cada paso del pipeline y las decisiones técnicas que determina.
- Entender cómo el pipeline RAG se integra con la ventana de contexto del modelo.

Dos fases, no una

El error más común al pensar en RAG es imaginarlo como una operación única: "el sistema busca en los documentos y el modelo responde". En realidad, RAG es un pipeline con dos fases que ocurren en momentos y contextos completamente diferentes.

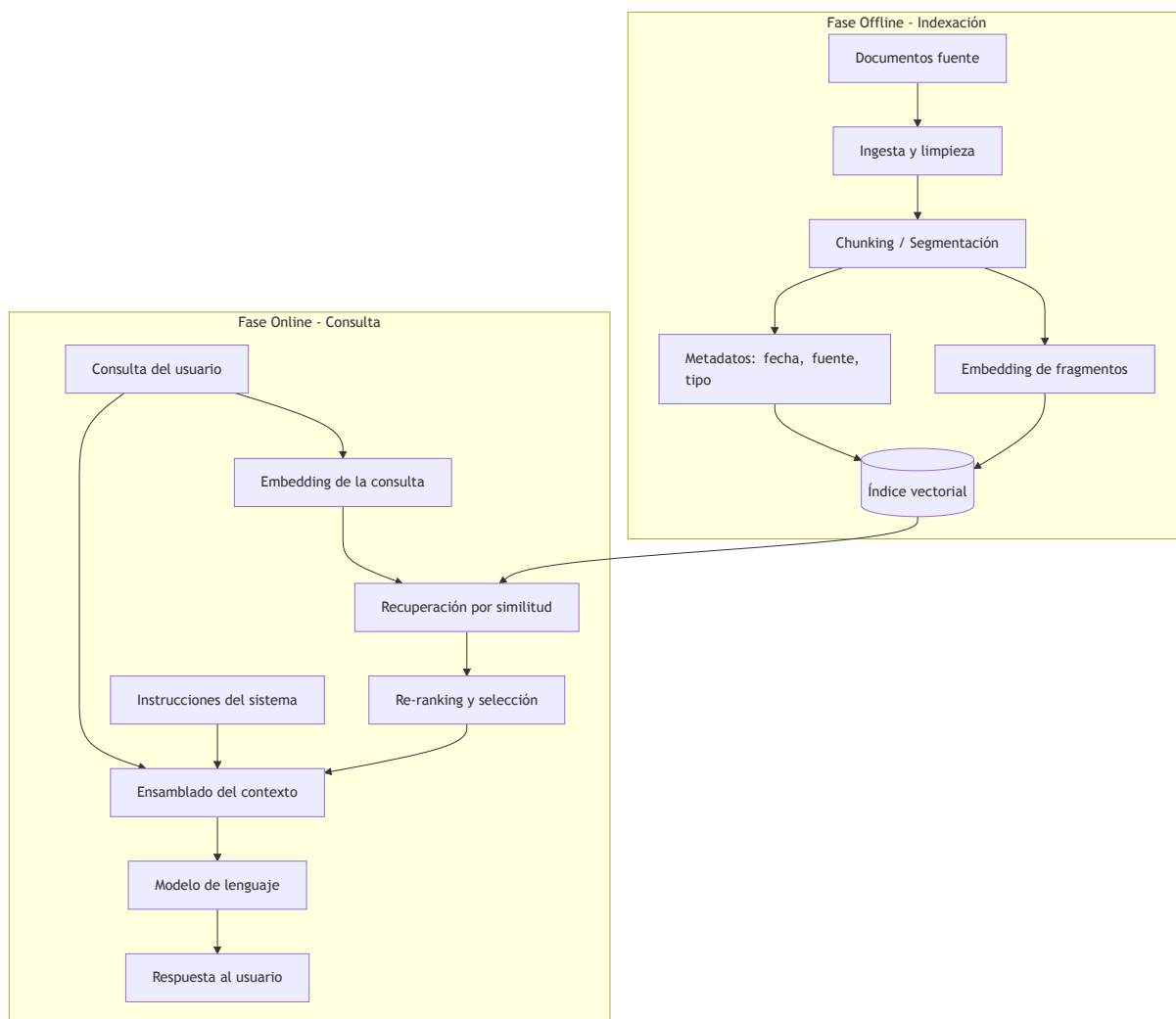
La **fase offline** —también llamada fase de indexación— ocurre antes de que cualquier usuario haga cualquier consulta. En esta fase el sistema ingiere documentos, los segmenta, convierte cada segmento en una representación matemática y lo almacena en una base

vectorial. Esta fase puede tardar segundos o días, dependiendo del volumen de documentos. Su resultado es un índice disponible para consulta.

La **fase online** —también llamada fase de recuperación e inferencia— ocurre en tiempo real, cuando el usuario hace una consulta. En esta fase el sistema convierte la consulta en una representación matemática, busca en el índice los fragmentos más similares, reordena los resultados si corresponde, ensambla el contexto aumentado y lo entrega al modelo para que genere la respuesta.

Entender esta distinción es importante para el diseño: los errores de indexación afectan a todas las consultas futuras. Un documento mal segmentado durante la fase offline generará recuperaciones deficientes durante meses hasta que se detecte y corrija el problema.

El pipeline completo



Fase offline: los pasos de indexación

1. Ingesta y limpieza

El primer paso es recolectar los documentos que formarán la base de conocimiento. Esta operación parece trivial pero presenta decisiones no triviales: ¿qué documentos se incluyen?, ¿cómo se manejan los formatos (PDF, Word, HTML, markdown)?, ¿se conservan las tablas y las imágenes o solo el texto?, ¿cómo se tratan los encabezados y los pies de página?

La limpieza incluye eliminar contenido redundante, artefactos de formato y secciones sin valor informativo (como páginas de créditos o disclaimers estándar repetidos en todos los documentos).

2. Chunking / Segmentación

Esta es la decisión más crítica de toda la fase offline y la que más impacto tiene sobre la calidad del retrieval. El modelo de embedding tiene una longitud máxima de texto que puede procesar. Los documentos suelen ser más largos. El chunking divide cada documento en fragmentos que quepan dentro de esa ventana.

Los tres enfoques principales son:

Chunking por tamaño fijo. Se corta cada N tokens o caracteres, independientemente de la estructura del texto. Es sencillo de implementar pero puede romper ideas en medio de una oración, degradando la coherencia del fragmento.

Chunking semántico. Se corta en límites naturales del texto: párrafos, secciones, artículos. Produce fragmentos más coherentes pero de tamaño variable, lo que puede complicar la estimación del uso de la ventana de contexto.

Chunking jerárquico. Se producen dos niveles de índice: uno con fragmentos pequeños (para recuperación precisa) y otro con fragmentos grandes (para contexto ampliado). El retrieval usa los fragmentos pequeños, pero al momento de insertar en el contexto se pueden recuperar los fragmentos grandes circundantes para añadir coherencia.

El solapamiento (overlap) es una variante que agrega N tokens del final de un fragmento al comienzo del siguiente, reduciendo el riesgo de que una idea quede partida entre dos fragmentos consecutivos.

3. Embedding de fragmentos

Cada fragmento se convierte en un vector numérico usando un modelo de embedding. Este vector captura el "significado" del fragmento en un espacio matemático de alta dimensión. Fragmentos con significados similares producen vectores cercanos en ese espacio.

La elección del modelo de embedding tiene consecuencias directas sobre la calidad del retrieval. Los modelos especializados en un idioma o dominio suelen superar a los modelos generales para ese dominio.

4. Almacenamiento en el índice vectorial

Los vectores —junto con los metadatos asociados (fuente, fecha, sección, tipo de documento)— se almacenan en una base vectorial que permite búsquedas por similitud eficientes.

Fase online: los pasos de recuperación

1. Embedding de la consulta

Cuando el usuario formula una consulta, esa consulta pasa por el mismo modelo de embedding que procesó los documentos. Este paso produce un vector que representa el significado de la consulta en el mismo espacio vectorial que los fragmentos del índice.

Es crítico que la consulta y los documentos sean procesados por el mismo modelo de embedding, o por modelos diseñados para ser compatibles. Si no lo son, los vectores no son comparables.

2. Recuperación por similitud

El sistema busca en el índice los fragmentos cuyo vector sea más cercano al vector de la consulta. La "cercanía" se mide con funciones de similitud como la similitud coseno o el producto punto. El resultado es un conjunto de candidatos ordenados por relevancia estimada.

La cantidad de candidatos recuperados —el parámetro k en "top-k retrieval"— es una decisión de diseño que equilibra calidad con costo: más candidatos aumentan la probabilidad de incluir el fragmento correcto, pero también la cantidad de ruido que el modelo debe procesar.

3. Re-ranking y selección

Los candidatos recuperados por similitud vectorial no son necesariamente los mejores candidatos para la consulta. La similitud semántica es una buena señal inicial, pero puede mejorarse con criterios adicionales. Esta etapa se desarrolla en detalle en la Sección 07.

4. Ensamblado del contexto

Los fragmentos seleccionados, la consulta del usuario y las instrucciones del sistema se combinan en el prompt que el modelo recibirá. El diseño de este ensamblado determina cómo el modelo "verá" la información recuperada: si los fragmentos van antes o después de la consulta, si se etiquetan con su fuente, si se ordenan por relevancia descendente.

5. Inferencia y respuesta

El modelo genera la respuesta basándose en el contexto ensamblado. Dado que ese contexto incluye fragmentos relevantes del índice, el modelo puede responder sobre información que

no tenía en su conocimiento interno, citar fuentes, comparar documentos y razonar sobre datos actualizados.

La ventana de contexto como restricción de diseño

El contexto que el sistema entrega al modelo tiene un límite: la ventana de contexto del modelo. Esta restricción determina cuántos fragmentos se pueden incluir en cada inferencia. Si el sistema recupera fragmentos demasiado largos o demasiado numerosos, el contexto superará el límite y el sistema deberá truncar, seleccionar o comprimir.

El diseño del chunking y la política de selección de fragmentos deben anticipar este límite. Una práctica común es estimar el presupuesto de tokens disponible en la ventana — descontando las instrucciones del sistema y la consulta del usuario— y seleccionar los fragmentos de mayor relevancia hasta completar ese presupuesto.

Ideas clave

- RAG tiene dos fases bien diferenciadas: indexación (offline) y recuperación (online).
 - El chunking es la decisión más crítica de la fase offline: define la granularidad de la recuperación.
 - La calidad del embedding determina qué tan bien el sistema puede relacionar consultas con fragmentos relevantes.
 - El contexto ensamblado por RAG compite por espacio en la ventana de contexto junto con las instrucciones del sistema y el historial conversacional.
 - Los errores en la fase offline afectan a todas las consultas futuras: la indexación debe tratarse con el mismo rigor que el diseño del modelo de inferencia.
-

Transición hacia la siguiente sección

El paso más técnico del pipeline RAG es la transformación de texto en vectores numéricos: el embedding. Sin comprender cómo funciona esa transformación, no es posible tomar decisiones informadas sobre la calidad del retrieval. La siguiente sección introduce los

embeddings como representación semántica y explica qué hace que dos vectores sean "cercaños" en el espacio que importa.

"Un arquitecto no memoriza respuestas. Comprende problemas para poder diseñar soluciones."

Módulo 3 — Context Engineering

Capítulo 06 — RAG (Retrieval-Augmented Generation) como componente del Context Engineering

Sección 04 — Embeddings y representación semántica

"Un embedding no representa palabras. Representa posiciones en un espacio donde el significado tiene geometría."

Objetivos de aprendizaje

- Comprender qué es un embedding y cómo convierte texto en una representación matemática.
 - Interpretar la noción de similitud semántica como cercanía geométrica en un espacio vectorial.
 - Identificar las propiedades que hacen a un modelo de embedding adecuado para RAG.
 - Evaluar criterios de selección de modelos de embedding para aplicaciones específicas.
-

El problema de representar significado

Para que un sistema pueda comparar documentos y encontrar los más relevantes para una consulta, necesita representar tanto los documentos como la consulta en un formato comparable. Las computadoras no comprenden texto de la misma manera que los humanos: operan sobre números.

La representación más directa de texto en números es la representación léxica: convertir cada palabra en un identificador numérico. Pero esta representación ignora el significado. Las

palabras "auto" y "vehículo" son completamente distintas en la representación léxica, aunque sean sinónimos en la mayoría de los contextos. Las palabras "banco" (institución financiera) y "banco" (asiento) son idénticas en la representación léxica, aunque tengan significados completamente distintos.

Los embeddings resuelven este problema. Un embedding es una representación densa de texto como un vector de números reales —típicamente entre 256 y 4096 dimensiones— donde las posiciones no son arbitrarias: textos con significados similares producen vectores cercanos en el espacio, y textos con significados distintos producen vectores alejados.

El espacio semántico

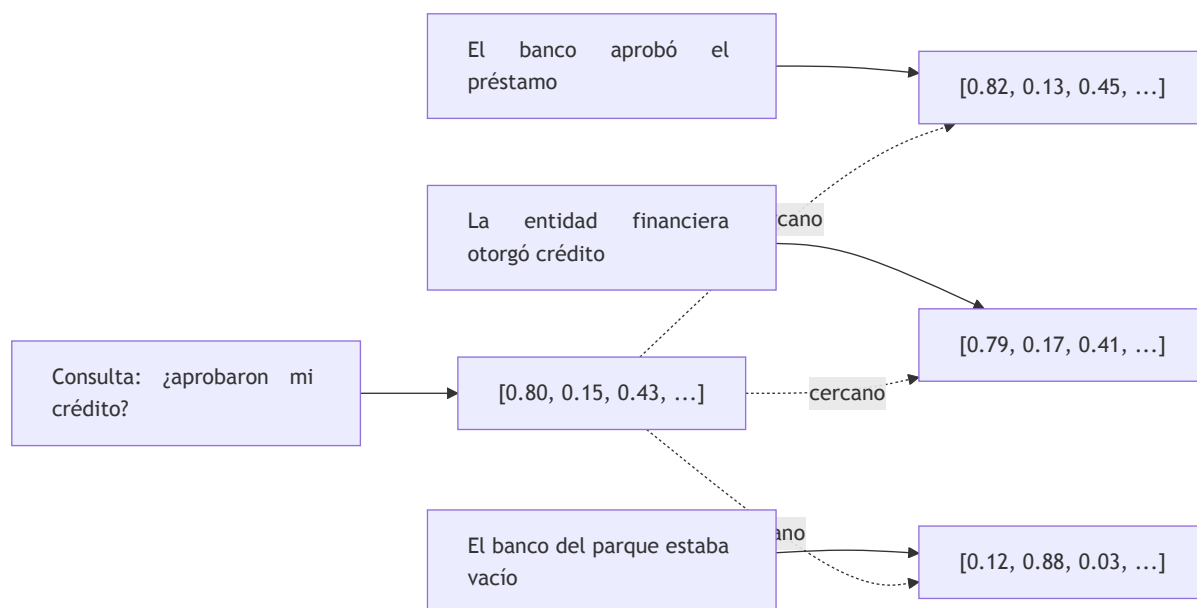
La idea central es que el significado puede representarse como posición en un espacio matemático. Ese espacio tiene muchas dimensiones —cientos o miles— y cada dimensión captura algún aspecto del significado, aunque esas dimensiones no sean interpretables individualmente de manera directa.

Lo que sí es interpretable es la geometría del espacio. Consideremos tres textos:

- "El banco aprobó el préstamo."
- "La entidad financiera otorgó crédito al solicitante."
- "El banco del parque estaba vacío."

En un espacio semántico bien construido, los vectores de los dos primeros textos estarán cerca entre sí —tienen el mismo significado sobre instituciones financieras— y ambos estarán lejos del tercero, que habla de un objeto de mobiliario urbano.

Esta propiedad de la geometría semántica es lo que hace posible el retrieval por similitud: para encontrar fragmentos relevantes a una consulta, basta encontrar los vectores más cercanos al vector de la consulta.



Cómo se producen los embeddings

Los modelos de embedding son redes neuronales entrenadas específicamente para producir esta representación geométrica del significado. No son los mismos modelos que generan texto: son modelos cuya salida no es texto sino un vector numérico.

El entrenamiento de un modelo de embedding se basa en pares de textos con alguna relación de similitud o disimilitud conocida. El modelo aprende a producir vectores cercanos para textos similares y vectores alejados para textos disímiles. Los datos de entrenamiento pueden provenir de pares pregunta-respuesta, de documentos con sus resúmenes, de oraciones consecutivas en el mismo documento, o de pares marcados manualmente.

El resultado es un modelo que, dado cualquier texto como entrada, produce un vector que "posiciona" ese texto en el espacio semántico aprendido.

Propiedades prácticas de los embeddings

Dimensionalidad. Los modelos de embedding producen vectores de dimensión fija. Modelos más simples trabajan con 256 o 384 dimensiones. Modelos más avanzados pueden llegar a 3072. Mayor dimensionalidad suele implicar mayor capacidad para capturar matices

semánticos, pero también mayor costo de almacenamiento y de cómputo durante la búsqueda.

Longitud máxima de entrada. Cada modelo de embedding tiene una longitud máxima de texto que puede procesar en una sola operación, medida en tokens. Esta limitación es la razón técnica detrás del chunking: si un documento supera la longitud máxima del modelo de embedding, no puede ser procesado como un todo.

Multilingüismo. Algunos modelos de embedding han sido entrenados en múltiples idiomas y producen vectores comparables entre idiomas distintos. Esto permite, por ejemplo, recuperar un documento en español usando una consulta en inglés. Los modelos monolingües suelen superar a los multilingües en el idioma para el que fueron optimizados, pero son inflexibles ante mezclas de idiomas.

Dominio específico. Un modelo de embedding entrenado con textos generales puede no capturar bien el vocabulario técnico de dominios especializados. Un modelo entrenado con textos legales, médicos o financieros suele producir representaciones más útiles para retrieval en esos dominios. La selección del modelo de embedding debe considerar el dominio del índice que se construirá.

Similitud como distancia

Una vez que documentos y consultas están representados como vectores, la similitud entre ellos se mide como una función de la distancia o el ángulo entre esos vectores.

La métrica más usada en RAG es la **similitud coseno**, que mide el ángulo entre dos vectores independientemente de su magnitud. Dos vectores que apuntan en la misma dirección tienen similitud coseno de 1.0 (máxima similitud). Dos vectores perpendiculares tienen similitud coseno de 0.0. Dos vectores opuestos tienen similitud coseno de -1.0.

La similitud coseno es adecuada cuando los vectores están normalizados. Cuando no lo están, el **producto punto** (dot product) tiene en cuenta tanto la dirección como la magnitud, lo que puede ser apropiado en algunos contextos pero introduce sesgos cuando los fragmentos tienen longitudes muy diferentes.

La **distancia L2** (distancia euclidiana) también se usa, aunque es menos frecuente en RAG puro y más común cuando los vectores no están normalizados.

Consideraciones para la selección del modelo de embedding

La elección del modelo de embedding para una aplicación RAG no es trivial. Los criterios relevantes son:

Criterio	Preguntas clave
Idioma	¿El corpus es en español, inglés, multilingüe?
Dominio	¿Es texto general, legal, médico, técnico?
Longitud de los documentos	¿Los fragmentos son párrafos cortos o secciones largas?
Presupuesto computacional	¿Cuánto cuesta producir un embedding por fragmento?
Latencia requerida	¿El embedding se produce en tiempo real o en batch?
Privacidad	¿Los textos pueden enviarse a un servicio externo (API) o deben procesarse localmente?

Para aplicaciones en español, los modelos basados en arquitecturas multilingües como mE5 o LaBSE suelen ofrecer resultados sólidos. Para inglés, los modelos de la familia text-embedding de OpenAI, los modelos de Cohere o los modelos de código abierto como all-MiniLM o BGE son opciones bien establecidas. Los benchmarks públicos como MTEB (Massive Text Embedding Benchmark) permiten comparar modelos en tareas de retrieval específicas.

Nota del Arquitecto

Muchos equipos seleccionan el modelo de embedding por popularidad o por disponibilidad en la plataforma que ya usan, sin evaluar si ese modelo es adecuado para el idioma y el dominio del corpus. El error tiene consecuencias invisibles en la etapa de diseño y costosas en producción: el sistema parece funcionar, pero el retrieval está sistemáticamente recuperando fragmentos subóptimos. La forma de

detectarlo es construir un conjunto de evaluación con consultas representativas y medir precision@5 con distintos modelos antes de comprometerse con uno.

Ideas clave

- Un embedding convierte texto en un vector numérico donde la posición codifica el significado.
- Textos con significados similares producen vectores cercanos; textos con significados distintos producen vectores alejados.
- La similitud semántica se mide como distancia geométrica entre vectores, típicamente usando similitud coseno.
- La elección del modelo de embedding determina la calidad del retrieval y debe considerar idioma, dominio y restricciones operativas.
- La longitud máxima del modelo de embedding es la razón técnica que impone el chunking en la fase de indexación.

Transición hacia la siguiente sección

Los embeddings resuelven el problema de la representación. El siguiente problema es el almacenamiento y la búsqueda eficiente: ¿cómo encontrar los vectores más cercanos a una consulta dentro de un índice que puede tener millones de fragmentos? La siguiente sección introduce las bases vectoriales y los algoritmos que hacen posible esa búsqueda a escala.

"Un arquitecto no memoriza respuestas. Comprende problemas para poder diseñar soluciones."

Módulo 3 — Context Engineering

Capítulo 06 — RAG (Retrieval-Augmented Generation) como componente del Context Engineering

Sección 05 — Bases vectoriales y búsqueda por similitud

"Una base vectorial no almacena documentos. Almacena significados organizados para ser recuperados por proximidad."

Objetivos de aprendizaje

- Comprender qué es una base vectorial y en qué se diferencia de una base de datos tradicional.
- Entender cómo los algoritmos de búsqueda aproximada permiten escalar el retrieval a millones de fragmentos.
- Conocer las principales opciones de infraestructura de bases vectoriales y sus características.
- Evaluar criterios de selección de base vectorial según el contexto de la aplicación.

Por qué las bases de datos relacionales no resuelven este problema

El retrieval en RAG requiere encontrar, dentro de un índice de fragmentos, los que más se parecen —semánticamente— a una consulta. Esa operación de "encontrar los más parecidos" no existe en las bases de datos relacionales tradicionales.

Una base de datos relacional busca registros que coincidan exactamente con un valor o que satisfagan una condición booleana. Puede encontrar todos los contratos de un cliente específico, o todos los documentos del departamento legal con fecha posterior al 1 de enero, o todos los artículos que contienen la palabra "riesgo". Lo que no puede hacer es encontrar los documentos cuyo significado sea más cercano al de una frase que el usuario acaba de escribir, especialmente si esa frase no usa exactamente las mismas palabras que los documentos.

Una base vectorial está diseñada para exactamente ese tipo de búsqueda. Almacena vectores y provee operaciones de consulta que retornan los vectores más cercanos al vector de consulta, según la métrica de similitud configurada.

La búsqueda exacta y sus límites

La forma más directa de encontrar el vector más cercano a una consulta es calcular la distancia entre el vector de consulta y cada uno de los vectores almacenados, y retornar los k más cercanos. Esta operación se llama búsqueda de vecinos más cercanos exacta (kNN exacto, por k-Nearest Neighbors).

El problema es la escala. Si el índice contiene un millón de fragmentos y cada vector tiene 768 dimensiones, calcular un millón de distancias para cada consulta es computacionalmente costoso. A medida que el índice crece, la latencia crece linealmente. Para índices de decenas de millones de fragmentos, la búsqueda exacta puede ser impracticable en tiempo real.

Las bases vectoriales modernas resuelven este problema con algoritmos de búsqueda aproximada.

Búsqueda aproximada: el equilibrio entre velocidad y precisión

Los algoritmos de búsqueda aproximada de vecinos más cercanos (ANN, por Approximate Nearest Neighbors) organizan los vectores durante la indexación de una manera que permite recuperar los k vecinos más probables sin comparar contra cada elemento del índice.

La "aproximación" implica un trade-off: la búsqueda puede no retornar el vecino exactamente más cercano en todos los casos, sino uno muy cercano. Para RAG, este compromiso es

habitualmente aceptable: la diferencia entre el fragmento más relevante y el segundo más relevante suele ser menor que el beneficio de poder responder en milisegundos en lugar de segundos.

Los dos algoritmos más difundidos en la práctica son:

HNSW (Hierarchical Navigable Small World). Organiza los vectores en un grafo jerárquico donde cada nodo está conectado a sus vecinos más cercanos en distintas escalas de resolución. La búsqueda navega desde el nivel más alto del grafo —con pocas conexiones globales— hasta el más bajo —con muchas conexiones locales— convergiendo rápidamente hacia los vecinos más cercanos al vector de consulta. HNSW ofrece alta velocidad y alta precisión, con el costo de mayor consumo de memoria durante la indexación.

IVF (Inverted File Index). Agrupa los vectores en clusters usando un algoritmo de cuantización como k-means. Cada cluster tiene un centroide. La búsqueda identifica primero los clusters más cercanos al vector de consulta y examina únicamente los vectores dentro de esos clusters. IVF consume menos memoria que HNSW pero puede ser menos preciso si el número de clusters examinados es bajo.

Filtrado por metadatos

Las bases vectoriales modernas no solo almacenan vectores: almacenan metadatos asociados a cada fragmento (fuente, fecha, autor, tipo de documento, nivel de acceso) y permiten combinar la búsqueda por similitud con filtros sobre esos metadatos.

Este filtrado es crítico en aplicaciones empresariales. Un sistema RAG que responde consultas de empleados de distintas divisiones no debería retornar fragmentos de documentos confidenciales a empleados sin los permisos correspondientes. Un sistema que maneja documentos con vigencia temporal no debería recuperar versiones obsoletas de una normativa.

El filtrado puede aplicarse antes de la búsqueda vectorial (pre-filtrado) o después (post-filtrado). El pre-filtrado reduce el conjunto de candidatos antes de calcular similitudes, lo que puede mejorar la eficiencia pero también puede reducir el recall si el filtro es muy restrictivo. El post-filtrado aplica el filtro sobre los k candidatos recuperados, lo que preserva el recall pero puede retornar menos de k resultados si muchos candidatos son filtrados.

Opciones de infraestructura

El ecosistema de bases vectoriales creció significativamente desde 2022. Las opciones más establecidas al momento de redactar este capítulo incluyen:

Sistema	Tipo	Características destacadas
pgvector	Extensión de PostgreSQL	Integración con ecosistema SQL, HNSW e IVF, adecuado para volúmenes moderados
Chroma	Base vectorial embebida	Ideal para prototipos y aplicaciones de un solo servicio, interfaz simple
Weaviate	Base vectorial distribuida	Búsqueda híbrida integrada, módulos para embedding automático
Qdrant	Base vectorial distribuida	Alto rendimiento, filtrado avanzado, soporte para búsqueda híbrida
Pinecone	SaaS administrado	Sin infraestructura a gestionar, latencia baja, costo variable por uso
Milvus	Base vectorial distribuida	Alta escala, soporte para múltiples índices simultáneos
Redis con Redisearch	Redis con módulo vectorial	Útil cuando Redis ya está en la arquitectura, latencia muy baja

La selección de la base vectorial depende de factores como el volumen de fragmentos, los requisitos de latencia, la disponibilidad de equipo para gestionar infraestructura, los requisitos de privacidad (local vs. SaaS) y la integración con el resto del stack tecnológico.

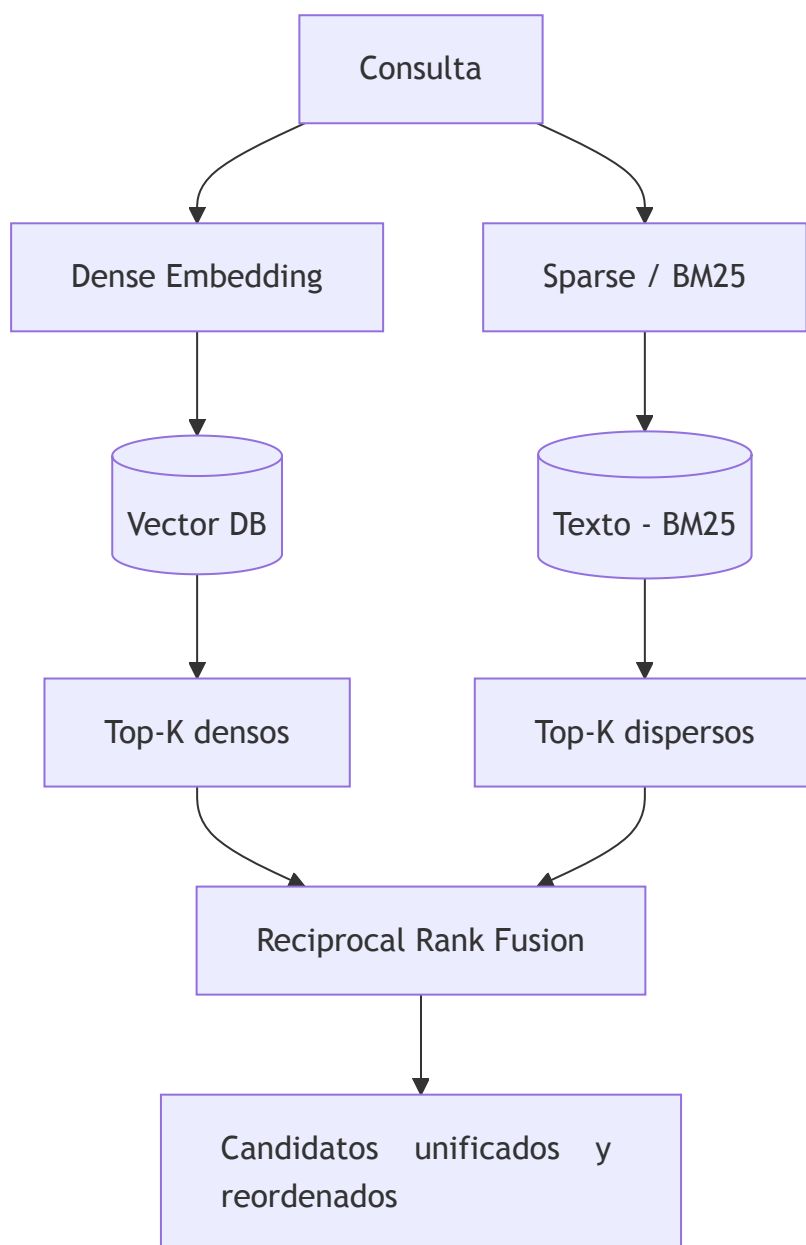
Búsqueda híbrida: combinando dense y sparse

La búsqueda vectorial pura (dense retrieval) basa la similitud en el significado global del texto. Pero a veces el retrieval precisa también de coincidencia exacta de términos: nombres propios, identificadores, siglas, números de artículo. Una búsqueda semántica puede no recuperar el fragmento correcto si la consulta incluye un número de contrato específico que el modelo de embedding no representa con suficiente precisión.

La búsqueda híbrida combina retrieval denso (vectorial) con retrieval disperso (basado en términos, típicamente BM25). BM25 es un algoritmo clásico de recuperación de información

que pondera la frecuencia de los términos de consulta en cada documento, ajustada por la longitud del documento. Es efectivo para recuperar documentos que contienen exactamente los términos buscados.

La combinación se realiza generalmente mediante una función de fusión de rankings como Reciprocal Rank Fusion (RRF), que combina las listas de resultados de ambos sistemas sin necesidad de conocer las puntuaciones absolutas de ninguno.



La búsqueda híbrida suele superar a cualquiera de los dos enfoques individuales en términos de recall. Weaviate y Qdrant incluyen soporte nativo para búsqueda híbrida. En otros sistemas, puede implementarse combinando dos pipelines de recuperación.

Ideas clave

- Las bases vectoriales están diseñadas para recuperar documentos por similitud semántica, no por coincidencia exacta de términos.
 - Los algoritmos de búsqueda aproximada (HNSW, IVF) permiten escalar el retrieval a millones de fragmentos manteniendo latencias bajas.
 - El filtrado por metadatos es esencial para implementar control de acceso y relevancia temporal en aplicaciones empresariales.
 - La búsqueda híbrida combina retrieval denso (vectorial) y disperso (BM25) para mejorar el recall en consultas que incluyen términos específicos.
 - La elección de la base vectorial debe considerar volumen, latencia, privacidad y capacidad operativa del equipo.
-

Transición hacia la siguiente sección

Con los embeddings y la base vectorial establecidos, el sistema puede recuperar candidatos. Pero la lista de candidatos que retorna la búsqueda vectorial no siempre es la lista óptima para incluir en el contexto del modelo. La siguiente sección examina las estrategias de recuperación avanzadas que mejoran la calidad de esa primera selección.

"Un arquitecto no memoriza respuestas. Comprende problemas para poder diseñar soluciones."

Módulo 3 — Context Engineering

Capítulo 06 — RAG (Retrieval-Augmented Generation) como componente del Context Engineering

Sección 06 — Estrategias de recuperación

"Recuperar los k fragmentos más cercanos es el punto de partida, no la solución final. La estrategia de recuperación define qué tan lejos llega el sistema de ese punto de partida."

Objetivos de aprendizaje

- Comprender las limitaciones del retrieval naive por similitud vectorial y las situaciones en que falla.
- Analizar estrategias de recuperación avanzadas: búsqueda con expansión de consulta, HyDE, MMR y recuperación multipasos.
- Evaluar cuándo aplicar cada estrategia en función del tipo de consulta y del corpus.
- Diseñar una política de recuperación que equilibre recall, precisión y costo computacional.

El retrieval naive y sus límites

La estrategia de recuperación más directa —calcular el embedding de la consulta y retornar los k fragmentos más cercanos— funciona sorprendentemente bien para consultas simples, directas y bien formuladas sobre un corpus coherente. Pero falla en situaciones habituales en aplicaciones reales.

Consultas vagas o cortas. "¿Cómo funciona el proceso?" es una consulta válida en muchos contextos, pero su vector de embedding captura poco significado discriminante. El retrieval retorna fragmentos genéricos en lugar de los específicos que el usuario necesita.

Consultas que usan vocabulario diferente al del corpus. Si el usuario pregunta "¿cuándo vence el contrato con el proveedor de infraestructura?" y los documentos usan "fecha de expiración del acuerdo de servicios de nube", la similitud semántica puede ser insuficiente si el modelo de embedding no captura bien esas equivalencias en el dominio.

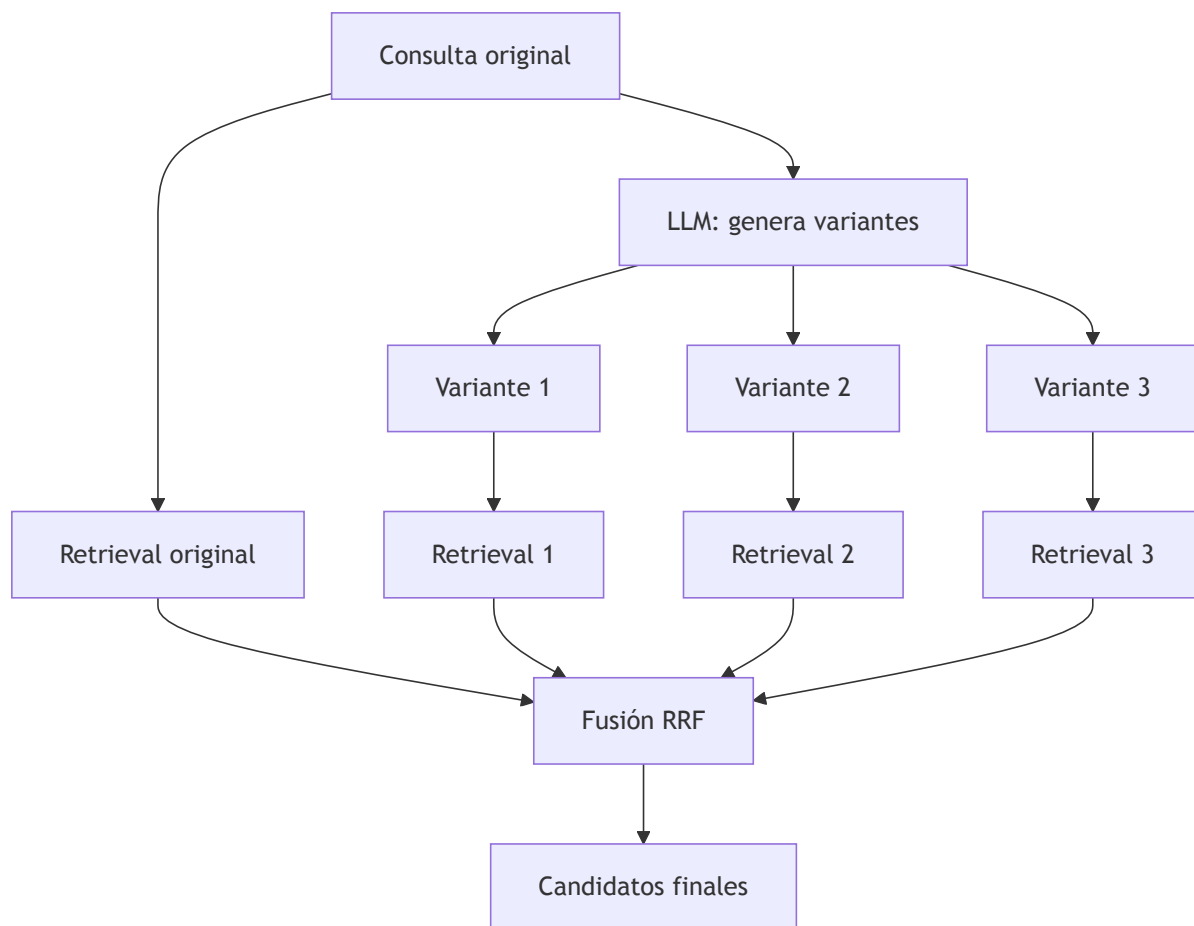
Consultas con múltiples sub-preguntas. "Dame los tres casos de uso más relevantes junto con sus costos estimados y tiempos de implementación" requiere recuperar información de distintas partes del corpus. Un único retrieval puede no cubrir todas las dimensiones.

Necesidad de diversidad. Si los k fragmentos más cercanos provienen todos del mismo documento o de la misma sección, el contexto puede ser redundante. El modelo recibe la misma información repetida en lugar de perspectivas complementarias.

Expansión de consulta

La expansión de consulta es la estrategia que amplía o reformula la consulta original antes de ejecutar el retrieval, para aumentar la probabilidad de recuperar fragmentos relevantes.

Una forma simple de expansión usa el propio LLM: se le pide que reformule la consulta en tres o cinco variantes diferentes, cada una desde un ángulo distinto. Luego se ejecuta el retrieval con cada variante y se fusionan los resultados usando Reciprocal Rank Fusion.



Esta técnica es especialmente efectiva cuando el usuario no sabe exactamente qué términos usa el corpus, o cuando la consulta es ambigua y puede interpretarse de varias maneras legítimas.

El costo es mayor: se ejecutan N veces el embedding y el retrieval, y se hace una llamada adicional al LLM. Para aplicaciones donde la latencia es crítica, puede ser un factor limitante.

HyDE: Hypothetical Document Embeddings

HyDE es una estrategia que aborda el problema de las consultas cortas o vagas de una manera diferente. En lugar de buscar directamente el embedding de la consulta en el índice, el sistema le pide al LLM que genere un documento hipotético que respondería a la consulta. Luego busca en el índice el fragmento más cercano a ese documento hipotético.

La lógica es que un documento hipotético tiene mayor longitud y vocabulario que una consulta corta, y por lo tanto su embedding captura más señal semántica. Si el LLM genera un

párrafo sobre el tema de la consulta —aunque ese párrafo sea inventado y potencialmente incorrecto en los detalles— ese párrafo se parece más a los fragmentos reales del corpus que una consulta de cinco palabras.

```
Consulta: "¿Por qué falla el sistema en producción?"
```

```
Documento hipotético generado por LLM:
```

```
"Los sistemas distribuidos en producción pueden fallar por diversas causas:  
agotamiento de recursos (memoria, CPU, conexiones de base de datos), latencia  
de red excesiva entre servicios, timeouts mal configurados, o condiciones de  
carrera en operaciones concurrentes. La monitorización proactiva mediante  
métricas de health check y alertas por percentiles de latencia permite  
detectar degradaciones antes de que se conviertan en fallos totales."
```

```
→ El embedding de este párrafo recupera fragmentos más específicos del corpus  
que el embedding de la consulta original.
```

HyDE mejora el recall para consultas vagas pero introduce un riesgo: si el documento hipotético contiene afirmaciones incorrectas sobre el dominio, puede sesgar el retrieval hacia fragmentos que no son los más relevantes. Su efectividad depende de la calidad del LLM para generar documentos coherentes con el dominio.

Maximum Marginal Relevance (MMR)

MMR es una estrategia de selección de fragmentos que combina relevancia con diversidad. En lugar de seleccionar los k fragmentos más similares a la consulta, selecciona los fragmentos que maximizan simultáneamente la similitud con la consulta y la diferencia entre sí.

El algoritmo es iterativo: en cada paso selecciona el fragmento que mejor equilibra ser relevante para la consulta y ser distinto de los fragmentos ya seleccionados. Un parámetro λ controla el equilibrio entre relevancia y diversidad: con $\lambda = 1.0$ se comporta como retrieval puro por similitud; con $\lambda = 0.0$ maximiza la diversidad ignorando la relevancia.

MMR es especialmente útil cuando:

- El corpus tiene muchos fragmentos muy similares entre sí (por ejemplo, múltiples versiones del mismo documento).
 - La consulta requiere perspectivas distintas para ser respondida bien.
 - El presupuesto de contexto es limitado y se quiere maximizar la información útil por token.
-

Recuperación multipasos (multi-hop retrieval)

Algunas consultas no pueden responderse con un único retrieval porque la información necesaria está distribuida en múltiples fragmentos no directamente relacionados entre sí. Para responder "¿cuál es el presupuesto del departamento que lidera el proyecto con mayor impacto en 2024?", el sistema necesita primero identificar el proyecto con mayor impacto, luego identificar el departamento que lo lidera, y finalmente recuperar el presupuesto de ese departamento. Cada paso requiere información del anterior.

La recuperación multipasos encadena varios ciclos de retrieval donde el resultado de cada ciclo informa la consulta del siguiente. Esto puede implementarse con un agente que orquesta la recuperación o con un pipeline que genera consultas intermedias basadas en los fragmentos recuperados en cada paso.

La complejidad y la latencia aumentan significativamente con cada salto. Para la mayoría de las aplicaciones empresariales, dos pasos son suficientes. Más de tres pasos suelen indicar que el problema debería resolverse con una estrategia de agentes, no con RAG puro.

Consideraciones de diseño para la política de recuperación

La elección de la estrategia de recuperación no debe ser universal: debe ajustarse al tipo de consulta y al corpus.

Escenario	Estrategia recomendada
Consultas directas sobre corpus coherente	Retrieval simple top-k
Consultas vagas o cortas	HyDE o expansión de consulta
Corpus con documentos similares entre sí	MMR
Consultas que mezclan términos exactos y semántica	Búsqueda híbrida (dense + BM25)
Consultas que requieren encadenar información	Multi-hop retrieval
Corpus muy grande con distribución desigual	IVF con expansión de consulta

En aplicaciones reales, lo más común es combinar varias estrategias: búsqueda híbrida para el retrieval inicial, MMR para la selección de candidatos, y expansión de consulta para consultas que el sistema detecta como vagas.

Nota del Arquitecto

El retrieval naive falla silenciosamente. El sistema sigue respondiendo, pero responde basándose en fragmentos subóptimos. Esta es la razón por la que los equipos que no miden la calidad del retrieval independientemente de la calidad de la respuesta final pueden creer que su sistema RAG funciona bien durante semanas, hasta que un caso extremo expone el problema. Medir recall@5 y precision@5 sobre un conjunto de consultas conocidas antes de desplegar cualquier cambio en la estrategia de recuperación es una práctica no opcional.

Ideas clave

- El retrieval naive por similitud vectorial es el punto de partida, no la solución final.
- La expansión de consulta y HyDE mejoran el retrieval para consultas cortas o ambiguas, a costo de latencia adicional.
- MMR balancea relevancia y diversidad, maximizando la información útil dentro del presupuesto de contexto.

- La búsqueda híbrida (dense + sparse) mejora el recall para consultas con términos específicos.
 - La política de recuperación debe diseñarse para el tipo de consulta predominante en la aplicación, no para un escenario genérico.
-

Transición hacia la siguiente sección

Las estrategias de recuperación mejoran el conjunto de candidatos. Pero incluso con buenos candidatos, el orden en que se presentan al modelo importa, y algunos candidatos que parecen relevantes pueden no serlo al analizar el contexto completo de la consulta. La siguiente sección introduce el re-ranking como la etapa que refina la selección final antes de ensamblar el contexto.

"Un arquitecto no memoriza respuestas. Comprende problemas para poder diseñar soluciones."

Módulo 3 — Context Engineering

Capítulo 06 — RAG (Retrieval-Augmented Generation) como componente del Context Engineering

Sección 07 — Re-ranking y selección de contexto

"El primer retrieval busca candidatos. El re-ranking elige cuáles merecen entrar al contexto del modelo."

Objetivos de aprendizaje

- Comprender por qué el retrieval inicial produce candidatos que requieren refinamiento antes de ser insertados en el contexto.
- Analizar las tres estrategias principales de re-ranking: semántico, temporal y por perfil de usuario.
- Diseñar una política de selección de contexto que maximice la utilidad dentro del presupuesto de tokens disponible.
- Identificar los trade-offs entre precisión del re-ranking y latencia de la respuesta.

Por qué el retrieval inicial no es suficiente

La búsqueda por similitud vectorial es eficiente y escalable, pero evalúa la relevancia de cada fragmento de manera independiente, sin considerar el conjunto. Esta evaluación independiente introduce dos tipos de problemas.

Problema de representación. Los modelos de embedding producen representaciones de significado global del texto. Son menos sensibles a matices locales: si la consulta menciona

una condición específica ("¿cuál es el plazo de entrega cuando el pedido supera 100 unidades?"), el fragmento más relevante puede no ser el que tiene mayor similitud coseno global, sino el que específicamente contiene esa condición. Los modelos de cross-encoding, que procesan consulta y fragmento juntos, capturan estos matices mejor que los modelos bi-encoder usados para el retrieval inicial.

Problema de conjunto. Si los k fragmentos recuperados provienen de la misma sección de un mismo documento, el contexto que el modelo recibirá será redundante. Un re-ranking que considera el conjunto, en lugar de cada fragmento individualmente, puede producir un contexto más rico con el mismo número de tokens.

El re-ranking es la etapa que aborda ambos problemas: toma los candidatos del retrieval inicial y los reordena o filtra aplicando criterios más sofisticados.

Re-ranking por relevancia semántica: los cross-encoders

Un cross-encoder es un modelo que recibe la consulta y un fragmento concatenados, y produce una puntuación de relevancia. A diferencia del bi-encoder usado para producir embeddings (que procesa consulta y fragmento por separado), el cross-encoder procesa ambos textos juntos, lo que permite capturar interacciones entre términos de la consulta y del fragmento.

Esta capacidad de capturar interacciones mejora significativamente la precisión del ranking, especialmente para consultas con condiciones específicas o terminología técnica. El costo es mayor latencia y costo computacional: un cross-encoder se ejecuta una vez por cada par (consulta, fragmento), lo que hace que su costo escale linealmente con el número de candidatos.

El patrón típico es:

1. Retrieval inicial con bi-encoder: recuperar los top 20-50 candidatos en milisegundos.
2. Re-ranking con cross-encoder sobre esos 20-50 candidatos: proceso más lento pero sobre un conjunto ya reducido.
3. Seleccionar los top 3-5 mejor puntuados por el cross-encoder para insertar en el contexto.

Modelos de re-ranking de código abierto como los de la familia BGE Reranker o Cohere Rerank están disponibles para distintos idiomas y dominios.

Re-ranking por actualidad temporal

En dominios donde la información cambia con frecuencia —normativas, precios, versiones de software, noticias de mercado—, un fragmento técnicamente relevante pero desactualizado puede ser más dañino que no tener ningún fragmento: el modelo lo presentará como verdadero aunque ya no lo sea.

El re-ranking temporal ajusta la puntuación de cada candidato según la distancia entre la fecha del documento y el momento de la consulta. Fragmentos de documentos recientes reciben una bonificación; fragmentos de documentos antiguos reciben una penalización.

La función de ajuste puede ser lineal (puntuación reducida N% por cada mes de antigüedad) o exponencial (puntuación reducida a la mitad cada semestre). La elección depende del ritmo de cambio del dominio: en normativa tributaria puede ser suficiente penalizar documentos de más de dos años; en precios de mercado puede ser necesario penalizar documentos de más de dos días.

```
Puntuación ajustada = Puntuación semántica × Factor_temporal
```

```
Factor_temporal:
```

- Documento de hoy: 1.0
- Documento de 6 meses: 0.85
- Documento de 1 año: 0.70
- Documento de 2 años: 0.50
- Documento de 3+ años: 0.25

La implementación requiere que los metadatos del índice incluyan la fecha de creación o última modificación del documento. Los sistemas RAG que no almacenan este metadato no pueden aplicar re-ranking temporal.

Re-ranking por perfil de usuario

En aplicaciones donde distintos usuarios tienen roles, preferencias o contextos de trabajo diferentes, los fragmentos óptimos para la misma consulta pueden variar según quién pregunta.

Un director financiero y un analista junior pueden hacer la misma consulta sobre el presupuesto del trimestre, pero el director puede necesitar el resumen ejecutivo mientras el analista necesita el detalle por línea de partida. Un usuario habitual de la sede de Madrid puede necesitar la versión de los procedimientos aplicable a España, no la versión global.

El re-ranking por perfil combina la puntuación semántica con señales del perfil del usuario:

- **Rol en la organización:** ciertos tipos de documentos son más relevantes para ciertos roles.
- **Historial de consultas:** fragmentos de documentos que el usuario consultó frecuentemente en el pasado pueden recibir una bonificación.
- **Ubicación o división:** documentos específicos a la región o departamento del usuario tienen prioridad sobre documentos globales cuando ambos son igualmente relevantes.
- **Idioma preferido:** en corpus multilingüe, fragmentos en el idioma nativo del usuario pueden recibir preferencia cuando existen equivalentes en distintos idiomas.

El re-ranking por perfil requiere que el sistema mantenga información sobre el usuario y que esa información esté disponible durante la fase de recuperación. En aplicaciones con requerimientos de privacidad estrictos, el alcance del perfil de usuario puede estar limitado por regulación.

Selección del contexto: el presupuesto de tokens

Una vez que el re-ranking produce una lista ordenada de fragmentos, el sistema debe decidir cuántos incluir en el contexto del modelo. Esta decisión está determinada por el presupuesto de tokens disponible en la ventana de contexto.

El presupuesto se calcula como:

```
Presupuesto de tokens para fragmentos =  
    Capacidad total de la ventana  
    - Tokens de las instrucciones del sistema  
    - Tokens del historial conversacional relevante  
    - Tokens de la consulta del usuario  
    - Margen reservado para la respuesta esperada
```

La selección greedy toma los fragmentos ordenados por puntuación de re-ranking y los añade al contexto hasta agotar el presupuesto. Esta estrategia es simple y efectiva en la mayoría de los casos.

Una variante más sofisticada usa programación dinámica para seleccionar la combinación de fragmentos que maximiza la puntuación total dentro del presupuesto, en lugar de simplemente tomar los más puntuados en orden. Esta variante puede mejorar la calidad cuando hay fragmentos de puntuación moderada pero complementaria entre sí.

El orden de los fragmentos en el contexto importa

La investigación en el comportamiento de los modelos de lenguaje ha documentado el fenómeno llamado "lost in the middle": los modelos tienden a prestar más atención al contenido al inicio y al final del contexto, y menos al contenido en el medio. Si el fragmento más relevante se inserta al final de una lista larga de fragmentos, el modelo puede "perderlo" al generar la respuesta.

La práctica recomendada es ordenar los fragmentos por relevancia decreciente y, si el modelo tiene esta tendencia conocida, colocar los más relevantes al principio del bloque de contexto. Algunos sistemas también añaden etiquetas explícitas ("Fragmento 1 (más relevante):", "Fragmento 2:", etc.) para guiar la atención del modelo.

Nota del Arquitecto

En muchos sistemas RAG que analicé en producción, el re-ranking estaba completamente ausente. El sistema recuperaba los top 5 por similitud vectorial y los insertaba directamente en el contexto, sin ningún criterio adicional. La mejora al agregar un cross-encoder de re-ranking era visible en las evaluaciones: precision@3 subía entre 15 y 30 puntos porcentuales dependiendo del corpus. El costo adicional — unos 100-200 ms por consulta— era completamente aceptable para las aplicaciones en cuestión. Si hay una sola optimización que recomendaría en un sistema RAG que ya funciona pero cuyas respuestas no son del todo precisas, es agregar un modelo de re-ranking.

Ideas clave

- El re-ranking mejora la selección de candidatos usando criterios más sofisticados que la similitud vectorial global.
 - Los cross-encoders capturan interacciones entre consulta y fragmento que los bi-encoders no pueden, a costo de mayor latencia.
 - El re-ranking temporal es crítico en dominios donde la información cambia frecuentemente; requiere metadatos de fecha en el índice.
 - El re-ranking por perfil personaliza el contexto según el rol, historial y contexto del usuario.
 - El orden de los fragmentos en el contexto impacta sobre qué presta atención el modelo al generar la respuesta.
-

Transición hacia la siguiente sección

Las secciones anteriores cubrieron los componentes técnicos individuales: embeddings, bases vectoriales, estrategias de recuperación y re-ranking. La siguiente sección los integra en una visión de conjunto: cómo se diseña y opera un pipeline RAG completo en un entorno empresarial real, con todas las consideraciones de producción que los tutoriales básicos omiten.

"Un arquitecto no memoriza respuestas. Comprende problemas para poder diseñar soluciones."

Módulo 3 — Context Engineering

Capítulo 06 — RAG (Retrieval-Augmented Generation) como componente del Context Engineering

Sección 08 — Diseño de un pipeline RAG empresarial

"En producción, RAG no es recuperar tres fragmentos y pasarlos al modelo. Es un sistema con política de ingesta, gestión de acceso, indexación incremental y monitoreo de relevancia."

Objetivos de aprendizaje

- Diseñar un pipeline RAG completo considerando todas las etapas de producción.
- Comprender las decisiones de chunking, solapamiento y enriquecimiento de metadatos en función de los requisitos del negocio.
- Aplicar políticas de control de acceso a nivel de documento durante el retrieval.
- Diseñar una estrategia de actualización incremental del índice que mantenga la relevancia a lo largo del tiempo.
- Identificar las métricas de monitoreo que indican degradación en la calidad del retrieval.

De los componentes al sistema

Las secciones anteriores introdujeron los componentes del pipeline RAG de forma aislada. En una aplicación empresarial, esos componentes deben ensamblarse en un sistema coherente que cumpla requisitos no técnicos: seguridad, trazabilidad, actualización continua, escalabilidad y observabilidad.

Un pipeline RAG empresarial no es solo la suma de sus partes técnicas. Es la suma de las decisiones de diseño que determinan cómo esas partes se comportan bajo carga real, con usuarios reales, sobre datos que cambian.

Decisiones de diseño en la fase de ingesta

Política de chunking

La primera decisión es la estrategia de segmentación. Para la mayoría de los corpus empresariales, el chunking semántico —dividir en párrafos o secciones lógicas— produce mejores resultados que el chunking por tamaño fijo. Sin embargo, requiere más trabajo de preprocesamiento, especialmente para documentos mal estructurados.

Una heurística práctica:

- Documentos con estructura clara (manuales, normativas, contratos): chunking por sección o artículo.
- Documentos narrativos o densos (informes, papers): chunking por párrafo con solapamiento del 10-15%.
- Documentos tabulares (hojas de cálculo, reportes con tablas): tratamiento especial que preserva las relaciones entre filas y columnas.

El tamaño del chunk debe estar calibrado al rango de tokens que el modelo de embedding puede procesar eficientemente, que para los modelos más comunes es entre 256 y 512 tokens. Chunks más cortos producen fragmentos más precisos pero con menos contexto; chunks más largos proporcionan más contexto pero pueden diluir la señal semántica.

Solapamiento

El solapamiento (overlap) entre chunks consecutivos es una decisión que afecta la calidad del retrieval para preguntas cuya respuesta está en la frontera entre dos fragmentos. Un solapamiento del 10-20% del tamaño del chunk es adecuado para la mayoría de los casos. Más solapamiento aumenta el tamaño del índice y puede introducir redundancia excesiva.

Enriquecimiento de metadatos

Cada fragmento debe almacenarse junto con metadatos que permitan filtrado, re-ranking temporal y trazabilidad:

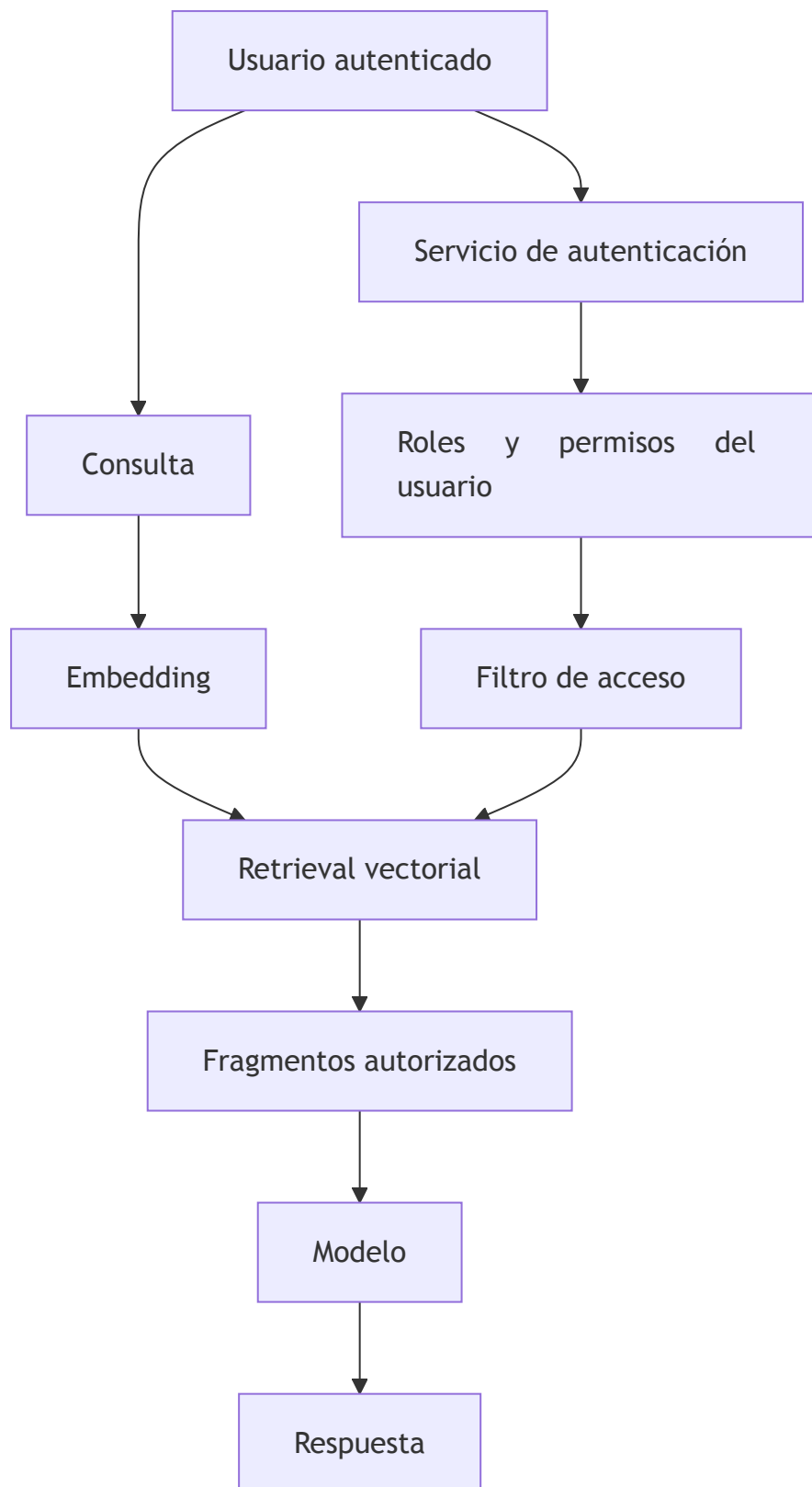
Metadato	Propósito
<code>fuelle</code>	URL o ruta del documento original
<code>fecha_creacion</code>	Para re-ranking temporal
<code>fecha_modificacion</code>	Detectar si el fragmento está desactualizado
<code>tipo_documento</code>	Política, manual, contrato, normativa, etc.
<code>division_organizacional</code>	Filtrado por división o departamento
<code>nivel_acceso</code>	Control de permisos de recuperación
<code>idioma</code>	Filtrado en corpus multilingüe
<code>version</code>	Para gestionar versiones de documentos que se actualizan

Control de acceso en el retrieval

En aplicaciones empresariales, los usuarios tienen distintos niveles de autorización para acceder a documentos. Un sistema RAG que no implementa control de acceso puede filtrar a un usuario documentos que no debería ver, aunque el sistema no lo muestre explícitamente: si un fragmento confidencial aparece en el contexto del modelo, el modelo puede parafrasearlo en su respuesta.

La implementación de control de acceso en RAG sigue el patrón de filtrado por metadatos:

1. Al indexar cada fragmento, se almacena su nivel de acceso o los roles que pueden verlo.
2. Al ejecutar el retrieval, la consulta se enriquece con el perfil de permisos del usuario autenticado.
3. El retrieval filtra candidatos cuyo nivel de acceso no coincide con los permisos del usuario.



El filtro puede aplicarse como pre-filtro (antes del retrieval, reduciendo el espacio de búsqueda) o como post-filtro (después del retrieval, descartando candidatos no autorizados). El pre-filtro es más eficiente pero puede afectar la calidad del retrieval si el espacio resultante

es muy pequeño. El post-filtro preserva la calidad pero puede retornar menos de k fragmentos cuando muchos candidatos son filtrados.

| Indexación incremental

Un índice RAG que se construye una vez y no se actualiza se convierte en un pasivo de información. Los documentos cambian, se crean nuevos documentos, algunos quedan obsoletos. Sin una política de actualización, el sistema sigue recuperando información que ya no es válida.

La indexación incremental es la estrategia que mantiene el índice sincronizado con el corpus fuente sin necesidad de reindexar desde cero cada vez.

Las operaciones necesarias son:

Agregar nuevos fragmentos. Cuando se incorpora un documento nuevo, se procesa (chunk, embed) y se agrega al índice sin tocar los fragmentos existentes.

Actualizar fragmentos de documentos modificados. Cuando un documento existente se actualiza, los fragmentos anteriores deben eliminarse del índice y los nuevos fragmentos deben agregarse. Esto requiere que cada fragmento tenga un identificador que permita relacionarlo con el documento fuente.

Eliminar fragmentos de documentos obsoletos. Cuando un documento se retira (una normativa derogada, un procedimiento discontinuado), sus fragmentos deben eliminarse del índice para evitar que sigan siendo recuperados.

Detección de contenido obsoleto. Para documentos que no tienen una fecha de retiro explícita, el sistema puede implementar políticas de caducidad automática basadas en antigüedad, o alertas que notifiquen a los administradores del índice sobre fragmentos que superan cierto umbral de antigüedad.

Una implementación práctica usa un registro de eventos (event log) de cambios en el corpus fuente. Un proceso periódico —o un trigger en tiempo real— procesa esos eventos y actualiza el índice en consecuencia.

Monitoreo de la calidad del retrieval

Un sistema RAG puede degradarse silenciosamente: el pipeline sigue funcionando, el modelo sigue generando respuestas, pero el índice ya no refleja la realidad del corpus y los fragmentos recuperados son cada vez menos relevantes.

Las métricas de monitoreo que permiten detectar esta degradación son:

Precision@k: De los k fragmentos recuperados, ¿qué proporción es realmente relevante para la consulta? Requiere un conjunto de evaluación con consultas anotadas.

Recall@k: ¿Qué proporción de los fragmentos relevantes existentes en el corpus es recuperada dentro de los top-k resultados?

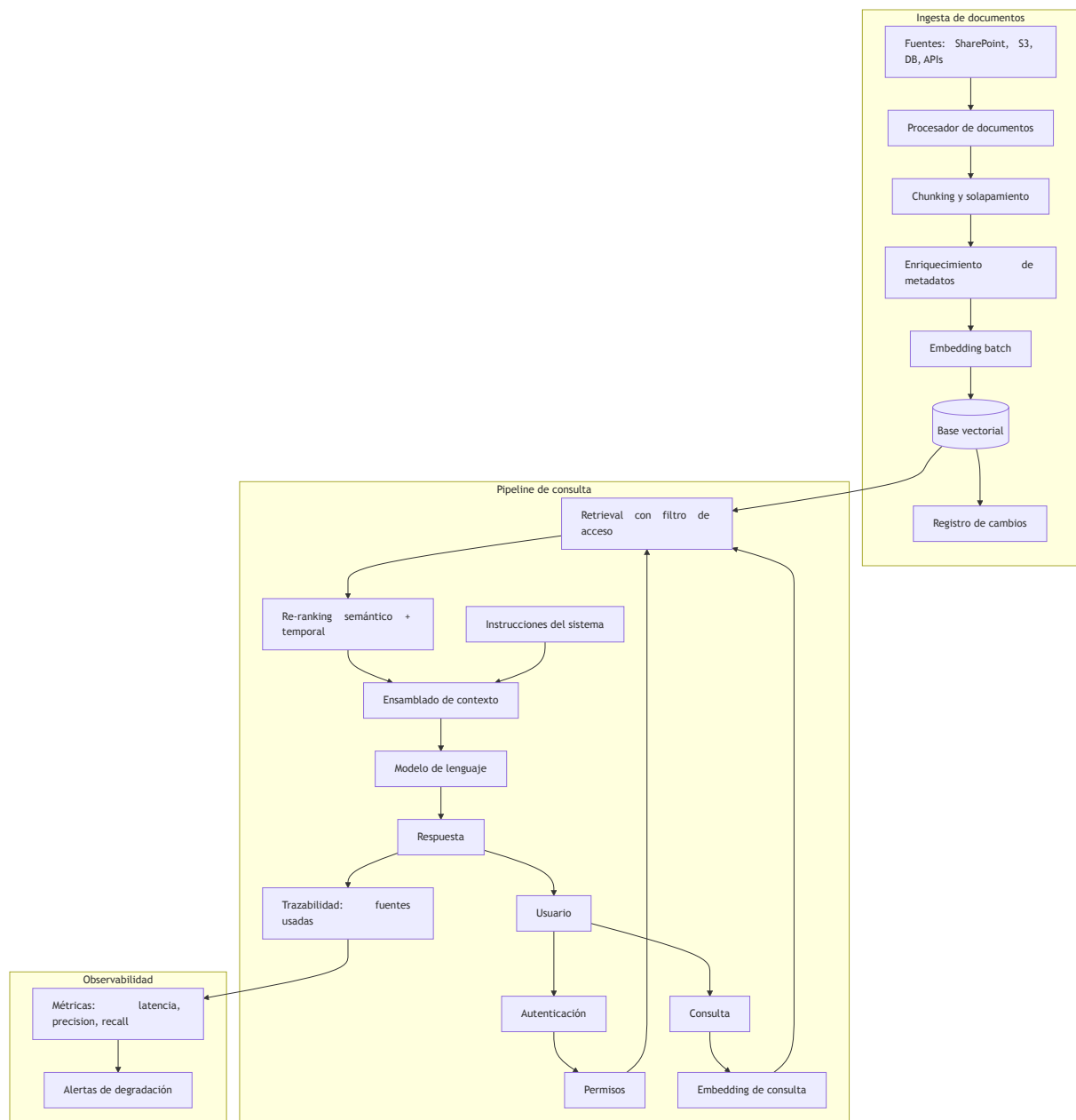
Mean Reciprocal Rank (MRR): ¿En qué posición aparece el primer fragmento relevante? MRR penaliza sistemas que recuperan el fragmento correcto pero lo ubican en posiciones bajas de la lista.

Tasa de respuestas "sin información suficiente": Si el modelo declara frecuentemente que no encontró información para responder, puede indicar que el retrieval no está encontrando los fragmentos correctos.

Latencia del pipeline completo: La latencia de extremo a extremo (desde la consulta hasta la respuesta) debe monitorearse por percentiles (p50, p95, p99), no solo por promedio. Un aumento en el p99 puede indicar problemas de indexación o sobrecarga del sistema de retrieval.

El conjunto de evaluación debe construirse con consultas representativas del uso real, con los fragmentos esperados marcados. Mantener y actualizar este conjunto es parte del trabajo operativo de un sistema RAG en producción.

Arquitectura de referencia para un pipeline empresarial



Ideas clave

- Un pipeline RAG empresarial incluye ingesta, control de acceso, indexación incremental, monitoreo y trazabilidad.
- El chunking y el enriquecimiento de metadatos son las decisiones de diseño con mayor impacto en la calidad del sistema a largo plazo.
- El control de acceso debe implementarse a nivel de fragmento, no solo a nivel de interfaz de usuario.

- La indexación incremental es un requisito operativo, no opcional: un índice estático se convierte en un pasivo de información.
 - Las métricas de retrieval (precision@k, recall@k, MRR) deben monitorearse independientemente de la calidad de la respuesta final del modelo.
-

Transición hacia la siguiente sección

Ahora que tenemos la arquitectura completa, podemos analizar qué funciona y qué no en la práctica. La siguiente sección cataloga los patrones de RAG que producen sistemas robustos y los anti-patrones que explican la mayoría de los fracasos en proyectos reales.

"Un arquitecto no memoriza respuestas. Comprende problemas para poder diseñar soluciones."

Módulo 3 — Context Engineering

Capítulo 06 — RAG (Retrieval-Augmented Generation) como componente del Context Engineering

Sección 09 — Patrones y anti-patrones de RAG

"Los patrones documentan lo que funciona en sistemas reales. Los anti-patrones documentan lo que inevitablemente falla, a veces después de meses de funcionamiento aparentemente correcto."

Objetivos de aprendizaje

- Identificar los patrones de diseño que producen sistemas RAG robustos y mantenibles.
- Reconocer los anti-patrones más frecuentes en proyectos reales y comprender por qué ocurren.
- Evaluar cuándo RAG es la solución correcta y cuándo el fine-tuning del modelo es preferible.
- Aplicar criterios de decisión arquitectónica al diseñar sistemas de recuperación.

Patrones de RAG

Patrón 1: RAG con filtrado por metadatos (Filtered RAG)

Descripción: El retrieval combina búsqueda por similitud vectorial con filtros explícitos sobre metadatos (fecha, tipo de documento, división organizacional, nivel de acceso).

Cuándo aplicarlo: Cuando el corpus contiene documentos con distintos ámbitos de aplicación y la relevancia de un fragmento depende tanto de su contenido como de sus atributos.

Ejemplo: Un sistema de consultas sobre procedimientos de RRHH donde los procedimientos varían por país. El retrieval filtra primero por `pais = "Argentina"` y luego aplica similitud vectorial dentro de ese subconjunto. Sin el filtro, el sistema podría recuperar el procedimiento correcto pero de la región equivocada.

Señal de que es necesario: Cuando los usuarios reportan respuestas correctas en contenido pero incorrectas en ámbito ("eso aplica a México, no a mí").

Patrón 2: RAG jerárquico (Hierarchical RAG)

Descripción: El índice se construye en dos niveles: un índice de resúmenes de alto nivel y un índice de fragmentos detallados. El retrieval usa primero el índice de resúmenes para identificar los documentos más relevantes, y luego recupera los fragmentos detallados de esos documentos.

Cuándo aplicarlo: Cuando el corpus contiene documentos largos y estructurados (informes extensos, manuales completos) donde el retrieval directo por fragmentos puede recuperar secciones correctas de documentos incorrectos.

Ejemplo: Un corpus de 500 informes anuales de distintas empresas. El retrieval de primer nivel identifica el informe de la empresa correcta. El retrieval de segundo nivel recupera los fragmentos específicos dentro de ese informe.

Beneficio: Mejora la coherencia del contexto: todos los fragmentos provienen de los documentos más relevantes para la consulta, no de una mezcla de documentos distintos.

Patrón 3: RAG con verificación de relevancia (Self-RAG)

Descripción: Después del retrieval, el sistema usa el propio LLM para evaluar si los fragmentos recuperados son relevantes para la consulta. Si la relevancia es insuficiente, el sistema reintenta el retrieval con una estrategia diferente antes de generar la respuesta.

Cuándo aplicarlo: Cuando la calidad de la respuesta es crítica y el corpus es heterogéneo, con documentos de muy distinta relevancia entre sí.

Ejemplo: Un asistente médico que, antes de responder una consulta sobre dosificación, verifica que los fragmentos recuperados correspondan a la indicación específica preguntada y no a contraindicaciones de otras indicaciones.

Costo: Introduce una llamada adicional al LLM antes de la generación, aumentando la latencia. No es adecuado para aplicaciones con restricciones estrictas de tiempo de respuesta.

Patrón 4: RAG con reescritura de consulta

Descripción: Antes del retrieval, el LLM reescribe la consulta del usuario en una forma más favorable para la búsqueda en el corpus específico.

Cuándo aplicarlo: Cuando el lenguaje de los usuarios difiere sistemáticamente del lenguaje de los documentos, o cuando las consultas son conversacionales y contienen referencias implícitas al contexto anterior.

Ejemplo: El usuario pregunta "¿y el plazo para el otro caso que mencionaste?" —una referencia implícita al historial conversacional. El sistema reescribe: "¿Cuál es el plazo de prescripción para reclamaciones por defectos ocultos en contratos de compraventa?"

Anti-patrones de RAG

Anti-patrón 1: RAG como sustituto de fine-tuning

Descripción: El equipo decide usar RAG para enseñarle al modelo cómo comportarse en el dominio —tono, vocabulario, formato de respuesta, criterios de evaluación—, indexando ejemplos de respuestas correctas como si fueran documentos de conocimiento.

Por qué falla: El comportamiento del modelo —cómo responde— es una función de su entrenamiento, no de su contexto. Insertar "ejemplos de cómo responder correctamente" en el índice no ajusta el comportamiento del modelo con la consistencia y la generalización que produce el fine-tuning. El contexto puede influir sobre la respuesta inmediata, pero esa influencia es frágil y no sistemática.

Cuándo el fine-tuning es preferible a RAG:

Criterio	RAG	Fine-tuning
El conocimiento cambia frecuentemente	Mejor	Peor
Se necesita trazabilidad de fuentes	Mejor	No disponible
Se requiere ajustar el tono/formato del modelo	No adecuado	Mejor
El corpus es estático y bien delimitado	Comparable	Mejor
Se necesita el modelo en producción sin latencia de retrieval	No adecuado	Mejor
El conocimiento es altamente especializado y estático	Comparable	Mejor

La regla práctica: RAG para datos que cambian o que necesitan trazabilidad; fine-tuning para ajustar comportamiento consistente del modelo.

Anti-patrón 2: Indexación sin política de actualización

Descripción: El equipo construye el índice una vez, sobre el corpus disponible en el momento del desarrollo, y lo despliega en producción sin ningún mecanismo para actualizarlo.

Por qué falla: Los documentos cambian. Normativas se actualizan, precios varían, procedimientos se revisan. Un índice estático convierte información desactualizada en respuestas con apariencia de veracidad. El sistema no solo no sabe la información nueva: afirma activamente la información vieja, con citas de fragmentos que el usuario puede interpretar como fuente de verdad.

Señal de alerta: Los usuarios reportan que el sistema cita documentos que ya no están vigentes, o que la información que provee difiere de la que encuentran en el repositorio oficial.

Solución: Diseñar la política de actualización desde el primer día, incluso si en el MVP solo se ejecuta manualmente una vez por semana. La política debe incluir cómo se detectan cambios, cómo se eliminan fragmentos obsoletos y cómo se agregan nuevos documentos.

Anti-patrón 3: $k=1$ o corpus sin diversidad de recuperación

Descripción: El sistema recupera un solo fragmento ($k=1$) o un número muy pequeño de fragmentos, asumiendo que el fragmento más similar es suficiente para responder la consulta.

Por qué falla: Muchas consultas requieren información distribuida en múltiples fragmentos. Con $k=1$, el modelo solo tiene acceso a una perspectiva parcial del tema. Además, el fragmento más similar no siempre es el más relevante: puede ser el más similar en vocabulario pero no el que contiene la información específica que la consulta necesita.

Consecuencia práctica: El modelo genera respuestas parciales o debe recurrir a su conocimiento interno para completar la respuesta, aumentando el riesgo de alucinación sobre los detalles que el fragmento no cubre.

Anti-patrón 4: Chunking descuidado

Descripción: Los fragmentos se dividen por tamaño fijo sin considerar la estructura del documento, partiendo definiciones en medio, separando condiciones de sus cláusulas base, o cortando tablas por la mitad.

Por qué falla: Un fragmento que comienza con "...en cuyo caso el plazo se extiende a 30 días" carece de contexto para determinar cuál es el caso y qué plazo está siendo extendido. El modelo recibirá ese fragmento como relevante para una consulta sobre plazos, pero no podrá usarlo correctamente porque el antecedente de la condición está en el fragmento anterior.

Señal de alerta: Las respuestas del modelo son temáticamente correctas pero factualmente incompletas; el modelo reconoce que "hay más información al respecto" pero no puede precisarla.

Anti-patrón 5: Ignorar la calidad del corpus

Descripción: El equipo indexa todos los documentos disponibles sin curaduría, incluyendo documentos obsoletos, borradores, versiones duplicadas, documentos escaneados con OCR de baja calidad y contenido irrelevante para el dominio de la aplicación.

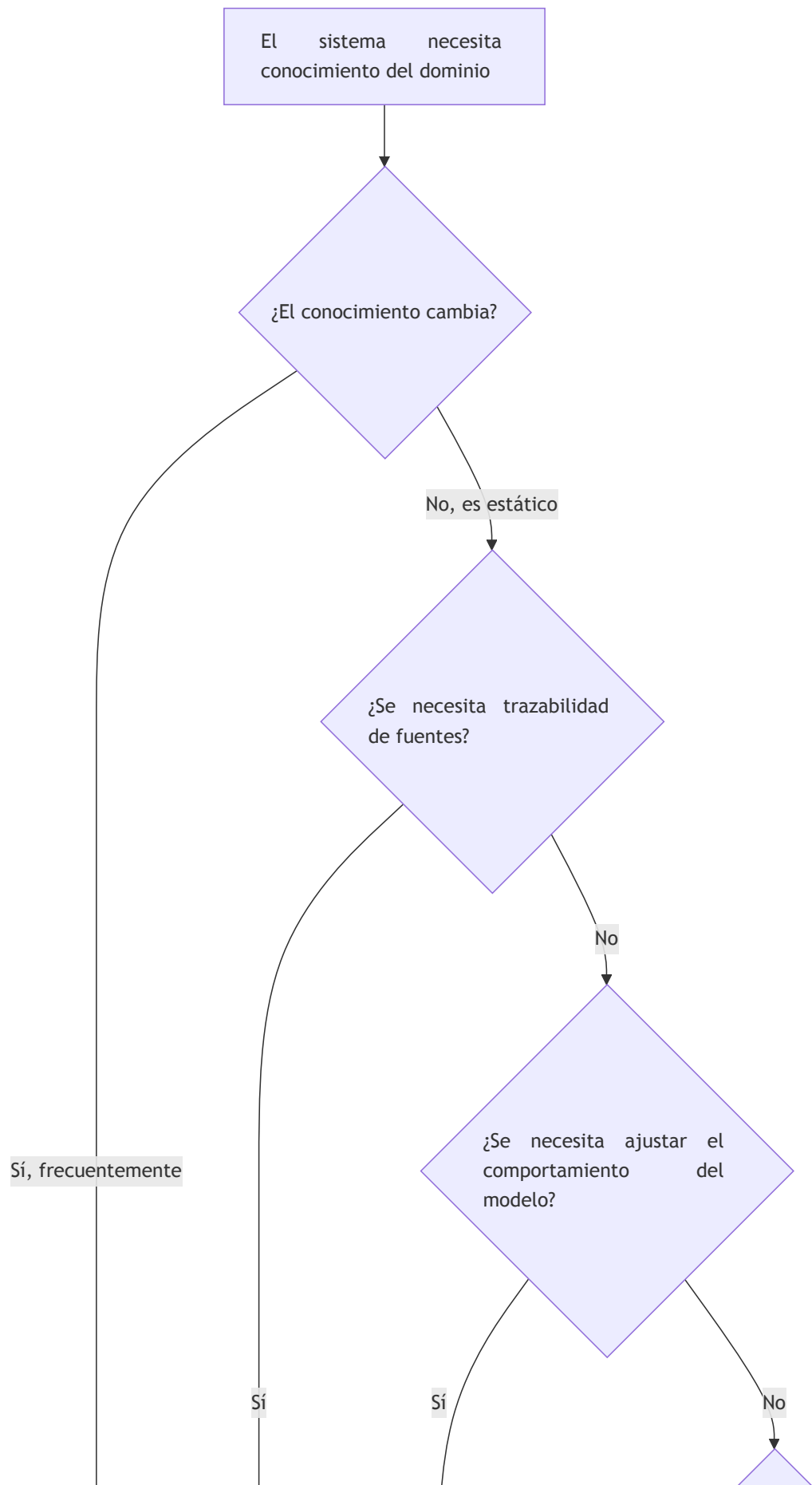
Por qué falla: El retrieval recupera fragmentos del corpus tal como existe. Si el corpus incluye basura, el retrieval recuperará basura con alta similitud semántica. El modelo recibirá fragmentos contradictorios (versión actualizada y versión obsoleta del mismo procedimiento),

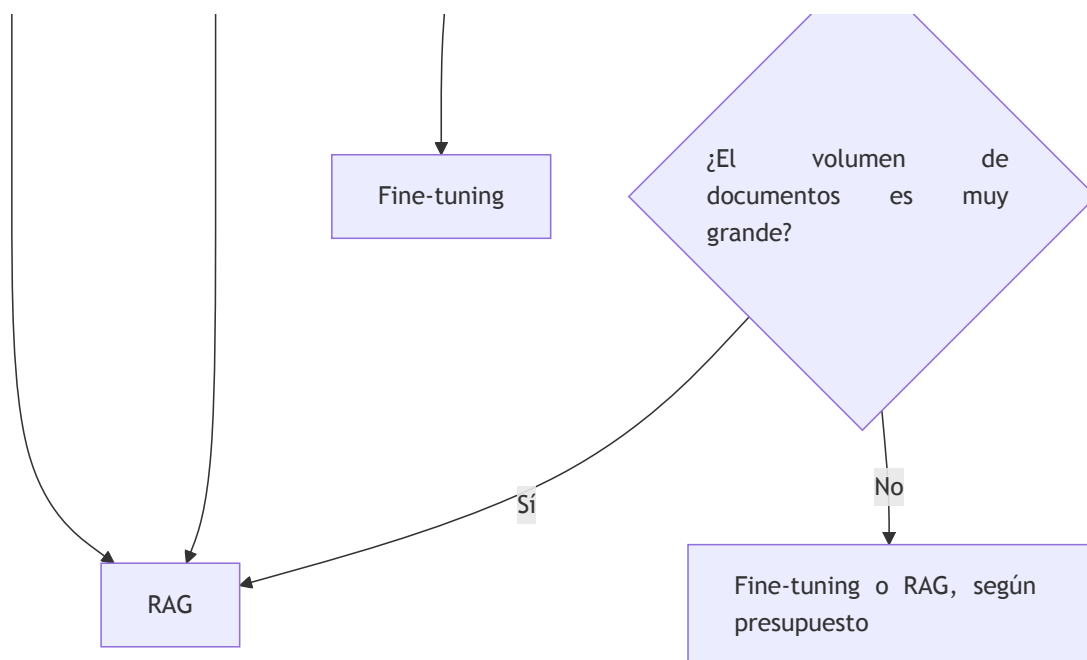
fragmentos sin sentido (artefactos de OCR) o fragmentos irrelevantes (documentos de otro dominio que coinciden superficialmente con la consulta).

Cómo tomar la decisión: ¿RAG o fine-tuning?

La pregunta que más frecuentemente debe responder un AI Engineer al diseñar un sistema de conocimiento es: ¿agrego RAG o hago fine-tuning?

El árbol de decisión simplificado:





En la práctica, muchos sistemas de producción usan ambas técnicas en conjunto: fine-tuning para ajustar el comportamiento base del modelo (tono, formato, estilo de razonamiento en el dominio) y RAG para proveer el conocimiento factual actualizable.

Ideas clave

- Los patrones de RAG más robustos combinan filtrado por metadatos, retrieval jerárquico y verificación de relevancia.
- RAG no es sustituto del fine-tuning; cada uno resuelve un problema diferente.
- Un índice sin política de actualización se convierte en un pasivo de información.
- La calidad del corpus es tan importante como la calidad del algoritmo de retrieval.
- $k=1$ es casi siempre insuficiente para aplicaciones reales.

Transición hacia la siguiente sección

Los patrones y anti-patrones describen situaciones genéricas. La siguiente sección materializa ese conocimiento en un caso de estudio completo: una aplicación empresarial diseñada de principio a fin, con justificación de cada decisión técnica.

"Un arquitecto no memoriza respuestas. Comprende problemas para poder diseñar soluciones."

Módulo 3 — Context Engineering

Capítulo 06 — RAG (Retrieval-Augmented Generation) como componente del Context Engineering

Sección 10 — Caso de estudio completo

"Un caso de estudio no es un tutorial. Es la demostración de que las decisiones de diseño tienen consecuencias concretas, y de que las mismas herramientas pueden producir sistemas excelentes o sistemas deficientes según cómo se use el criterio."

Objetivos de aprendizaje

- Aplicar los conceptos del capítulo a un escenario empresarial realista y concreto.
- Analizar y justificar cada decisión de diseño en función de los requisitos del negocio.
- Identificar las diferencias entre un diseño RAG naive y un diseño RAG robusto para el mismo problema.
- Comprender cómo los requisitos de acceso, actualización y trazabilidad moldean la arquitectura.

El problema: asistente de consultas normativas para una firma de servicios financieros

Una firma de servicios financieros con 800 empleados en cuatro países (Argentina, Chile, México y España) necesita un asistente interno que responda consultas sobre:

- Normativa regulatoria aplicable por país (CNBV, CNMV, CMF, CNV).
- Procedimientos internos de compliance.

- Políticas de riesgo de crédito.
- Contratos marco con contrapartes.

El contexto determina los requisitos:

1. La normativa cambia con frecuencia: actualizaciones regulatorias ocurren varias veces al año.
2. Los documentos tienen distintos niveles de confidencialidad: algunos contratos solo pueden verlos los socios; las políticas generales las puede ver todo el personal.
3. La normativa varía por país: un analista en Chile no debería recibir como primera respuesta la regulación de México.
4. La firma necesita trazabilidad: cuando el sistema cita una normativa, el compliance officer debe poder verificar qué versión del documento fue la fuente.

Decisión 1: ¿RAG o fine-tuning?

El corpus normativo cambia varias veces al año. La trazabilidad es un requisito explícito. La información es tanto pública (normativa regulatoria) como privada (contratos internos). Fine-tuning no produce trazabilidad y no puede actualizarse sin reentrenar.

Decisión: RAG como mecanismo principal de acceso al conocimiento. Fine-tuning no está descartado para una segunda fase donde se quiera ajustar el tono formal y la estructura de las respuestas del modelo, pero no es el punto de partida.

Decisión 2: Chunking

Los documentos del corpus tienen características distintas:

- **Normativa regulatoria:** documentos estructurados por artículos numerados. Cada artículo es una unidad semántica completa. → Chunking por artículo.
- **Procedimientos internos:** documentos en prosa con secciones numeradas. → Chunking semántico por sección, con solapamiento del 15% para preservar la coherencia entre secciones contiguas.
- **Contratos:** estructura legal con cláusulas. → Chunking por cláusula. Cada cláusula se almacena con referencia a las cláusulas que la condicionan (metadato

`clausulas_relacionadas`).

El tamaño máximo de chunk se establece en 400 tokens, compatible con el modelo de embedding seleccionado.

Decisión 3: Modelo de embedding

El corpus contiene documentos en español de cuatro variedades (rioplatense, chilena, mexicana, peninsular) y terminología legal y financiera especializada. Un modelo de embedding general en inglés no es adecuado.

Opciones evaluadas:

- **mE5-large (multilingual)**: buen rendimiento en español, entrenado con corpus multilingüe que incluye texto legal.
- **text-embedding-3-large de OpenAI**: alto rendimiento general, disponible vía API, texto enviado a terceros.
- **BGE-M3**: modelo de código abierto con soporte multilingüe, ejecutable localmente.

Dado que el corpus incluye contratos confidenciales que no deben enviarse a APIs externas, se selecciona **BGE-M3 ejecutado localmente**. El costo de infraestructura es mayor, pero el requisito de privacidad es no negociable.

Decisión 4: Base vectorial

Requisitos:

- Filtrado por metadatos (país, nivel de acceso, tipo de documento).
- Búsqueda híbrida (los usuarios a veces buscan por número de artículo exacto).
- Soporte para actualización incremental.
- Equipo de operaciones pequeño: no se puede gestionar un clúster distribuido complejo.

Decisión: Qdrant ejecutado en contenedor Docker. Soporta filtrado avanzado por metadatos, búsqueda híbrida (dense + sparse) y actualización incremental sin downtime. El volumen del corpus (estimado en 50.000 fragmentos) es manejable con una instancia única bien dimensionada.

Decisión 5: Metadatos del índice

Cada fragmento se indexa con:

```
{
  "fuente": "CNBV_Circular_2024_003_art_42",
  "tipo": "normativa",
  "pais": "MX",
  "organismo": "CNBV",
  "fecha_publicacion": "2024-03-15",
  "fecha_vigencia": "2024-04-01",
  "nivel_acceso": "todos",
  "version": "1.0",
  "clausulas_relacionadas": ["art_38", "art_41"]
}
```

Decisión 6: Estrategia de recuperación

El sistema implementa búsqueda híbrida porque los usuarios a veces preguntan por número de artículo exacto ("¿qué dice el artículo 42 de la circular CNBV?") y a veces por concepto ("¿qué requisitos de capital mínimo aplican a fondos de inversión?").

El retrieval aplica pre-filtro por `pais` (determinado a partir del perfil del usuario autenticado) y por `nivel_acceso` (determinado a partir de los roles del usuario). Sobre el subconjunto filtrado, se ejecuta búsqueda híbrida con Reciprocal Rank Fusion.

Se recuperan los top 20 candidatos, que pasan a la etapa de re-ranking.

Decisión 7: Re-ranking

Dos criterios de re-ranking se aplican en secuencia:

1. **Re-ranking semántico** con un cross-encoder fine-tuneado para texto legal en español (BGE Reranker). Produce una puntuación de relevancia más precisa que la similitud vectorial para consultas con condiciones específicas.

2. **Re-ranking temporal:** fragmentos de documentos publicados en los últimos 12 meses reciben una bonificación del 20%. Fragmentos de documentos de más de 36 meses reciben una penalización del 40%. Esto refleja la dinámica regulatoria del sector.

Los top 5 fragmentos tras el re-ranking se insertan en el contexto del modelo.

Decisión 8: Política de actualización del índice

La normativa regulatoria se actualiza mediante un proceso batch diario que consulta los repositorios oficiales de cada organismo regulatorio. Cuando se detecta un documento nuevo o modificado, el proceso:

1. Elimina del índice todos los fragmentos del documento anterior (identificados por `fFuente`).
2. Procesa el nuevo documento (chunking, embedding).
3. Agrega los nuevos fragmentos con los metadatos actualizados.
4. Registra el evento en el log de auditoría.

Los documentos internos se actualizan mediante webhook: cuando un documento se modifica en el sistema de gestión documental interno, se dispara la reindexación automática.

Decisión 9: Trazabilidad

Cada respuesta del sistema incluye, junto al texto generado, una lista de las fuentes utilizadas:

```
Respuesta: Los fondos de inversión que operen en México están sujetos a  
un requisito de capital mínimo de [X] según la normativa vigente...
```

```
Fuentes:
```

```
[1] CNBV Circular 2024-003, Art. 42 (publicado 2024-03-15, vigente desde 2024-04-01)
```

```
[2] CNBV Circular 2023-019, Art. 38 (publicado 2023-09-01)
```

Esta trazabilidad está implementada recuperando los metadatos de los fragmentos usados en el contexto y formateándolos como parte de la respuesta. El compliance officer puede verificar

que la cita corresponde a la versión correcta del documento.

Resultado: diferencias entre el diseño naive y el diseño robusto

Dimensión	Diseño naive	Diseño robusto
Chunking	Por tamaño fijo (500 chars)	Por artículo/sección/cláusula según tipo de documento
Filtrado por país	Ausente	Pre-filtro por país del usuario autenticado
Control de acceso	Ausente	Filtrado por nivel de acceso en el retrieval
Re-ranking	Ausente (top-k directo)	Cross-encoder + temporal
Actualización del índice	Manual, ad hoc	Batch diario + webhook
Trazabilidad	Ausente	Metadatos de fuente en cada respuesta
Búsqueda	Solo dense	Híbrida (dense + sparse)

La diferencia no está en el modelo de lenguaje —ambos sistemas usarían el mismo modelo— sino en las decisiones de ingeniería que rodean al modelo.

Ideas clave

- Los requisitos de negocio (privacidad, trazabilidad, multijurisdicción) determinan las decisiones de arquitectura RAG, no las preferencias tecnológicas.
- Un sistema RAG empresarial robusto no se diferencia del naive en el modelo sino en el diseño del pipeline completo.
- La política de actualización del índice debe diseñarse desde el primer día, no como una mejora futura.
- La trazabilidad de fuentes es un requisito frecuente en dominios regulados y debe planificarse desde la arquitectura.

Transición hacia la siguiente sección

El caso de estudio describe decisiones. La siguiente sección convierte esas decisiones en experiencia práctica: el laboratorio práctico propone construir un pipeline RAG mínimo viable, desde la ingesta hasta la evaluación de los fragmentos recuperados.

"Un arquitecto no memoriza respuestas. Comprende problemas para poder diseñar soluciones."

Módulo 3 — Context Engineering

Capítulo 06 — RAG (Retrieval-Augmented Generation) como componente del Context Engineering

Sección 11 — Laboratorio práctico

"Un pipeline RAG mínimo viable enseña más en dos horas de práctica que diez horas de lectura. El objetivo no es el código: es observar cómo cada decisión de diseño afecta la calidad del resultado."

Objetivo del laboratorio

Construir un pipeline RAG mínimo viable que cubra la totalidad del ciclo: desde la ingesta de documentos hasta la evaluación de los fragmentos recuperados para una consulta dada. No se requiere experiencia previa con bases vectoriales ni APIs de LLM: el laboratorio usa componentes de código abierto ejecutables localmente.

Al finalizar el laboratorio, el estudiante habrá:

- Segmentado un conjunto de documentos usando dos estrategias de chunking distintas.
- Producido embeddings y almacenado los fragmentos en un índice vectorial local.
- Ejecutado consultas y recuperado fragmentos.
- Evaluado manualmente la calidad del retrieval.
- Comparado dos configuraciones distintas del mismo pipeline.

Requisitos

- Python 3.10 o superior.

- Librerías: `chromadb`, `sentence-transformers`, `langchain-text-splitters` (o `nltk`).
- Acceso a un LLM: una cuenta en cualquier proveedor con API compatible (o un modelo local con Ollama).
- No se requiere GPU: los modelos de embedding de tamaño pequeño funcionan en CPU con latencia aceptable para este ejercicio.

Instalación:

```
pip install chromadb sentence-transformers langchain-text-splitters
```

Corpus del laboratorio

Para mantener el foco en el pipeline y no en la adquisición de datos, el laboratorio usa un corpus de 10 documentos de texto libre sobre un tema acotado. Se sugiere usar artículos de la Wikipedia en español sobre un dominio de interés (por ejemplo: 5 artículos sobre contratos en derecho civil y 5 artículos sobre regulación financiera). El corpus debe guardarse como archivos `.txt` en un directorio `/corpus`.

El ejercicio puede repetirse con cualquier corpus de documentos internos, lo que lo convierte en un prototipo directo para aplicaciones reales.

Paso 1: Segmentación con chunking fijo y chunking semántico

```
from langchain_text_splitters import RecursiveCharacterTextSplitter
import os

# Leer documentos
documentos = []

for filename in os.listdir("./corpus"):
    if filename.endswith(".txt"):
        with open(f"./corpus/{filename}", "r", encoding="utf-8") as f:
            documentos.append({"texto": f.read(), "fuente": filename})

# Chunking fijo: 500 tokens, solapamiento de 50
splitter_fijo = RecursiveCharacterTextSplitter(
    chunk_size=500,
    chunk_overlap=50,
    length_function=len
)

# Chunking semántico por párrafo: se divide en "\n\n"
splitter_semantico = RecursiveCharacterTextSplitter(
    separators=["\n\n", "\n", ".", " "],
    chunk_size=600,
    chunk_overlap=80,
    length_function=len
)

# Producir chunks con ambas estrategias
chunks_fijos = []
chunks_semanticos = []

for doc in documentos:
    for chunk in splitter_fijo.split_text(doc["texto"]):
        chunks_fijos.append({"texto": chunk, "fuente": doc["fuente"]})
    for chunk in splitter_semantico.split_text(doc["texto"]):
        chunks_semanticos.append({"texto": chunk, "fuente": doc["fuente"]})
```

```
print(f"Chunks fijos: {len(chunks_fijos)}")  
print(f"Chunks semánticos: {len(chunks_semanticos)}")
```

Observar la diferencia en el número de chunks y en la coherencia de los fragmentos generados. ¿Cuántos chunks del método fijo cortan ideas a mitad de oración?

Paso 2: Embedding e indexación

```
import chromadb

from sentence_transformers import SentenceTransformer

# Modelo de embedding (ejecuta en CPU, ~90MB)
modelo = SentenceTransformer("paraphrase-multilingual-MiniLM-L12-v2")

# Cliente de ChromaDB (base vectorial local)
cliente = chromadb.Client()

# Crear dos colecciones: una para cada estrategia de chunking
coleccion_fija = cliente.create_collection("chunks_fijos")
coleccion_semantica = cliente.create_collection("chunks_semanticos")

def indexar_chunks(chunks, coleccion):

    textos = [c["texto"] for c in chunks]

    fuentes = [c["fuente"] for c in chunks]

    ids = [f"chunk_{i}" for i in range(len(chunks))]

    # Generar embeddings en batch
    embeddings = modelo.encode(textos, batch_size=32, show_progress_bar=True)

    coleccion.add(
        documents=textos,
        embeddings=embeddings.tolist(),
        metadatas=[{"fuente": f} for f in fuentes],
        ids=ids
    )

    print(f"Indexados {len(chunks)} fragmentos en '{coleccion.name}'")

indexar_chunks(chunks_fijos, coleccion_fija)

indexar_chunks(chunks_semanticos, coleccion_semantica)
```

Paso 3: Retrieval y comparación

```

# Definir un conjunto de 5 consultas de prueba
consultas = [
    "¿Qué es un contrato de mutuo y cuáles son sus elementos esenciales?",
    "¿Cuáles son las sanciones por incumplimiento de normas de capital mínimo?",
    "¿En qué casos puede resolverse un contrato por causa de fuerza mayor?",
    "¿Qué diferencia hay entre una garantía personal y una garantía real?",
    "¿Cómo se regula la transparencia en los instrumentos financieros?"
]

def recuperar(consulta, coleccion, n_resultados=3):
    embedding_consulta = modelo.encode([consulta])[0].tolist()
    resultados = coleccion.query(
        query_embeddings=[embedding_consulta],
        n_results=n_resultados,
        include=["documents", "metadatas", "distances"]
    )
    return resultados

# Comparar para la primera consulta
for consulta in consultas[:2]:
    print(f"\n{'='*60}")
    print(f"CONSULTA: {consulta}")
    print(f"\n{'='*60}")

    print("\n--- Retrieval con chunking FIJO ---")
    res_fijo = recuperar(consulta, coleccion_fija)
    for i, (doc, dist) in enumerate(zip(res_fijo["documents"][0], res_fijo["distances"]):
        print(f"\n[{i+1}] Similitud: {1-dist:.3f}")
        print(doc[:200] + "...")

    print("\n--- Retrieval con chunking SEMÁNTICO ---")
    res_sem = recuperar(consulta, coleccion_semantica)
    for i, (doc, dist) in enumerate(zip(res_sem["documents"][0], res_sem["distances"]):

```

```
print(f"\n[{i+1}] Similitud: {1-dist:.3f}")

print(doc[:200] + "...")
```

Paso 4: Evaluación manual de la calidad del retrieval

Para cada una de las 5 consultas, registrar:

Consulta	Fragmento 1 relevante?	Fragmento 2 relevante?	Fragmento 3 relevante?	Estrategia ganadora
Consulta 1	✓/✗ (fijo) / ✓/✗ (sem)	...	...	fijo / semántico / empate
Consulta 2	...	...	...	...
Consulta 3	...	...	...	...
Consulta 4	...	...	...	...
Consulta 5	...	...	...	...

La evaluación es manual en este laboratorio. En producción, esta tabla se construye con anotaciones del dominio y se calcula automáticamente como precision@3.

Paso 5: Integración con LLM (opcional)

Si se dispone de acceso a una API de LLM, el pipeline puede completarse ensamblando el contexto y generando la respuesta:

```
# Ejemplo conceptual con cualquier API compatible con OpenAI
from openai import OpenAI

cliente_llm = OpenAI(api_key="tu_api_key")

def rag_completo(consulta, coleccion, n_resultados=3):
    # Retrieval
    resultados = recuperar(consulta, coleccion, n_resultados)
    fragmentos = resultados["documents"][0]
    fuentes = [m["fuente"] for m in resultados["metadatas"][0]]

    # Ensamblado del contexto
    contexto = "\n\n".join([
        f"[Fragmento {i+1} - Fuente: {fuente}]\n{fragmento}"
        for i, (fragmento, fuente) in enumerate(zip(fragmentos, fuentes))
    ])

    # Prompt
    prompt = f"""Responde la siguiente consulta basándote únicamente en los
fragmentos proporcionados. Si la información no es suficiente, indícalo.

CONSULTA: {consulta}

FRAGMENTOS RECUPERADOS:

{contexto}

RESPUESTA: """

    respuesta = cliente_llm.chat.completions.create(
        model="gpt-4o-mini",
        messages=[{"role": "user", "content": prompt}]
    )

    return respuesta.choices[0].message.content

# Probar con la primera consulta
```

```
respuesta = rag_completo(consultas[0], coleccion_semantica)
print(respuesta)
```

Preguntas para reflexionar después del laboratorio

1. ¿Qué diferencias observaste entre el chunking fijo y el semántico en la coherencia de los fragmentos? ¿Qué tipo de consulta favorece cada estrategia?
2. ¿Hubo alguna consulta donde ninguna estrategia recuperó un fragmento relevante en los top 3? ¿Qué podría explicarlo?
3. ¿Cómo cambiaría el pipeline si el corpus tuviera 50.000 documentos en lugar de 10? ¿Qué componentes deberían reemplazarse o ajustarse?
4. ¿Qué metadatos agregarías si este corpus fuera de documentos normativos de una organización real?
5. Si el modelo de embedding fuera reemplazado por uno más potente, ¿esperarías que el retrieval mejorara uniformemente para todas las consultas? ¿Por qué sí o por qué no?

Variantes opcionales

Variante A: Implementar búsqueda híbrida (dense + BM25) usando la librería `rank_bm25` y comparar los resultados con el retrieval puramente vectorial.

Variante B: Agregar un modelo de cross-encoding para re-ranking usando `cross-encoder/ms-marco-multilingual-MiniLM-L12` de sentence-transformers y medir el cambio en `precision@3`.

Variante C: Agregar filtrado por metadatos: clasificar los documentos como "legal" o "financiero" durante la indexación y comparar el retrieval con y sin el filtro aplicado a la consulta.

"Un arquitecto no memoriza respuestas. Comprende problemas para poder diseñar"

soluciones."

Módulo 3 — Context Engineering

Capítulo 06 — RAG (Retrieval-Augmented Generation) como componente del Context Engineering

Sección 12 — Checklist del AI Engineer

"Un checklist no reemplaza el criterio. Lo estructura para que el criterio no se pierda bajo la presión del proyecto."

Propósito

Este checklist está organizado en tres momentos del ciclo de vida de un sistema RAG: antes de construir, durante la implementación y en producción. No es exhaustivo: es el conjunto mínimo de preguntas que un AI Engineer debe poder responder en cada etapa. Si alguna respuesta es "no sé" o "no lo hemos definido", esa es la señal de que esa decisión requiere atención antes de avanzar.

Antes de construir

Validación del problema

- [] ¿El conocimiento que el sistema necesita está efectivamente fuera del modelo base (es reciente, privado o de nicho)?
- [] ¿La respuesta correcta depende de documentos específicos cuya fuente necesita ser trazable?
- [] ¿El conocimiento cambia con frecuencia suficiente como para que el fine-tuning sea impracticable?

- [] ¿Se evaluó si el problema puede resolverse con instrucciones del sistema bien diseñadas, sin necesidad de RAG?

Definición del corpus

- [] ¿Están definidos qué documentos forman parte del corpus y cuáles no?
- [] ¿El corpus tiene una fuente canónica de verdad (un repositorio oficial, un sistema de gestión documental)?
- [] ¿Se identificaron documentos obsoletos, duplicados o de baja calidad que deben excluirse?
- [] ¿Existe un proceso claro para agregar nuevos documentos al corpus?

Requisitos de seguridad y acceso

- [] ¿Hay documentos con distintos niveles de confidencialidad?
- [] ¿El sistema debe implementar control de acceso a nivel de fragmento?
- [] ¿El corpus puede enviarse a APIs externas o debe procesarse localmente por requisitos de privacidad?

Durante la implementación

Fase de indexación

- [] ¿Se evaluaron al menos dos estrategias de chunking (fija y semántica) sobre una muestra del corpus?
- [] ¿El tamaño del chunk es compatible con la longitud máxima del modelo de embedding?
- [] ¿Se definió el solapamiento y se justificó en función del tipo de documentos?
- [] ¿Los metadatos de cada fragmento incluyen fuente, fecha de creación/modificación, tipo de documento y nivel de acceso?
- [] ¿Se verificó que los fragmentos generados son coherentes (no parten ideas a mitad)?

Modelo de embedding

- [] ¿El modelo de embedding fue evaluado en el idioma y dominio del corpus específico?
- [] ¿Se consultaron benchmarks públicos (MTEB u otro) para comparar opciones?
- [] ¿La misma versión del modelo se usará tanto para indexar documentos como para procesar consultas?

Base vectorial

- [] ¿La base vectorial soporta filtrado por metadatos con la granularidad requerida?
- [] ¿El algoritmo de indexación (HNSW, IVF) está configurado para el volumen de fragmentos del corpus?
- [] ¿La base vectorial permite actualizaciones incrementales sin reindexación completa?
- [] ¿Se diseñó la estrategia de backup y recuperación del índice?

Estrategia de recuperación

- [] ¿Se evaluó si el retrieval naive (top-k por similitud) es suficiente o si se requieren estrategias adicionales (HyDE, expansión de consulta, MMR)?
- [] ¿Se implementa búsqueda híbrida (dense + sparse) cuando el corpus requiere coincidencia exacta de términos?
- [] ¿El valor de k (número de candidatos recuperados) está justificado en función del presupuesto de contexto?

Re-ranking y selección de contexto

- [] ¿Se implementa algún modelo de re-ranking (cross-encoder) o el retrieval vectorial es el criterio final?
- [] ¿El re-ranking temporal está habilitado para dominios con información que cambia frecuentemente?
- [] ¿El presupuesto de tokens para fragmentos está calculado correctamente, descontando instrucciones del sistema y reserva para la respuesta?
- [] ¿El orden de inserción de fragmentos en el contexto está optimizado para la atención del modelo?

Conjunto de evaluación

- [] ¿Existe un conjunto de consultas de evaluación con los fragmentos esperados marcados?
- [] ¿Se calculó precision@k y recall@k baseline antes de desplegar a producción?
- [] ¿El conjunto de evaluación cubre los tipos de consulta más frecuentes en la aplicación?

En producción

Política de actualización del índice

- [] ¿Existe un proceso automatizado para detectar cambios en el corpus fuente?
- [] ¿El proceso de actualización elimina correctamente los fragmentos obsoletos?
- [] ¿Se registra en un log de auditoría cada operación de agregar, modificar o eliminar fragmentos?
- [] ¿Existe un mecanismo para invalidar manualmente fragmentos que se detecten incorrectos?

Monitoreo de calidad

- [] ¿Se monitorea la latencia del pipeline completo (desde consulta hasta respuesta)?
- [] ¿Se registra qué fragmentos se recuperan para cada consulta (para análisis posterior)?
- [] ¿Existe una forma de detectar cuando el retrieval sistemáticamente no encuentra fragmentos relevantes?
- [] ¿Se calculan métricas de retrieval (precision@k, MRR) sobre muestras de producción periódicamente?

Trazabilidad y auditoría

- [] ¿La respuesta del sistema incluye referencia a las fuentes utilizadas?
- [] ¿Los logs permiten reconstruir qué fragmentos se insertaron en el contexto para una consulta específica?
- [] ¿Se almacenan los metadatos de versión del documento fuente junto con cada respuesta?

Seguridad

- [] ¿Se verifica en cada consulta que el usuario tiene permisos para ver los fragmentos recuperados?
- [] ¿El sistema previene que el contenido de fragmentos confidenciales aparezca en respuestas a usuarios no autorizados?
- [] ¿Se registran intentos de acceso a fragmentos fuera del nivel de autorización del usuario?

Señales de alerta en producción

Estas son situaciones que indican degradación del sistema y requieren investigación inmediata:

- **Usuarios reportan que la información está desactualizada:** el índice no se está actualizando correctamente.
- **Las respuestas son genéricas aunque el corpus tiene información específica:** el retrieval no está encontrando los fragmentos correctos.
- **El modelo declara frecuentemente "no tengo información suficiente":** k puede ser demasiado bajo, o el chunking produce fragmentos poco coherentes.
- **La latencia del pipeline aumenta progresivamente:** el índice creció sin reconfigurar los parámetros del algoritmo de búsqueda.
- **Respuestas contradictorias para la misma consulta:** el índice puede contener versiones distintas del mismo documento.

"Un arquitecto no memoriza respuestas. Comprende problemas para poder diseñar soluciones."

Módulo 3 — Context Engineering

Capítulo 06 — RAG (Retrieval-Augmented Generation) como componente del Context Engineering

Sección 13 — Resumen del capítulo

Lo que aprendimos en este capítulo

Este capítulo presentó RAG como una estrategia del Context Engineering para incorporar conocimiento externo al contexto que el modelo recibe en cada inferencia. El recorrido siguió una progresión deliberada: del problema, a los componentes técnicos, a la arquitectura completa, a los criterios de decisión.

El problema que RAG resuelve

El conocimiento interno de un modelo de lenguaje queda congelado en la fecha de corte del entrenamiento. Hay tres clases de conocimiento que el modelo no puede tener en su estado base: conocimiento temporal (posterior al corte), conocimiento privado (documentos internos de la organización) y conocimiento de nicho (dominio especializado subrepresentado en el preentrenamiento).

Actualizar el conocimiento vía reentrenamiento o fine-tuning es costoso, lento y no produce trazabilidad. RAG desacopla el conocimiento del sistema del conocimiento del modelo, permitiendo actualizar el primero sin modificar el segundo.

La arquitectura en dos fases

RAG tiene dos fases bien diferenciadas:

Fase offline (indexación): los documentos se ingresan, se segmentan en fragmentos (chunking), cada fragmento se convierte en un vector (embedding) y se almacena en una base vectorial con sus metadatos asociados. Los errores en esta fase afectan a todas las consultas futuras.

Fase online (recuperación e inferencia): la consulta del usuario se convierte en un vector, el sistema busca los fragmentos más similares, los reordena mediante re-ranking, ensambla el contexto y lo entrega al modelo para que genere la respuesta.

Los componentes técnicos

Embeddings: representaciones vectoriales del texto donde fragmentos con significados similares producen vectores cercanos en el espacio semántico. La elección del modelo de embedding debe considerar idioma, dominio y restricciones de privacidad.

Bases vectoriales: sistemas de almacenamiento y búsqueda diseñados para comparar vectores por similitud. Los algoritmos de búsqueda aproximada (HNSW, IVF) permiten escalar el retrieval a millones de fragmentos con latencias bajas. El filtrado por metadatos permite implementar control de acceso y relevancia temporal.

Las decisiones que determinan la calidad

Chunking: la estrategia de segmentación es la decisión de mayor impacto sobre la calidad del retrieval. Fragmentos que parten ideas incompletas producen recuperaciones irrelevantes aunque el algoritmo de búsqueda sea correcto.

Estrategia de recuperación: el retrieval naive (top-k por similitud vectorial) puede complementarse con expansión de consulta, HyDE, MMR o búsqueda híbrida (dense + BM25) según el tipo de consulta y el corpus.

Re-ranking: los cross-encoders mejoran la precisión del ranking sobre los candidatos iniciales. El re-ranking temporal es crítico en dominios con información dinámica. El re-ranking por perfil personaliza el contexto según el usuario.

Las decisiones que determinan la robustez

Política de actualización del índice: un índice estático se convierte en un pasivo de información. La actualización incremental es un requisito operativo, no una mejora futura.

Control de acceso a nivel de fragmento: el filtrado por nivel de acceso debe aplicarse durante el retrieval, no solo en la interfaz de usuario.

Trazabilidad de fuentes: en dominios regulados, cada respuesta debe incluir referencia a los documentos que la respaldaron.

Monitoreo de calidad: precision@k, recall@k y MRR deben medirse sobre consultas representativas, de forma continua en producción.

RAG dentro del Context Engineering

RAG es una de las cuatro estrategias del Context Engineering cubiertas en este módulo:

Estrategia	Función
Instrucciones del sistema	Define comportamiento, rol y restricciones
Memoria conversacional	Mantiene continuidad entre turnos y sesiones
Resumen y compresión	Gestiona el contexto cuando supera la ventana
RAG	Incorpora conocimiento externo relevante por consulta

En aplicaciones empresariales completas, estas cuatro estrategias coexisten y se coordinan dentro de la misma arquitectura.

La decisión RAG vs. fine-tuning

RAG es preferible cuando el conocimiento cambia frecuentemente o cuando se requiere trazabilidad. El fine-tuning es preferible cuando el objetivo es ajustar el comportamiento del modelo (tono, formato, estilo de razonamiento) de forma consistente. En sistemas de producción maduros, ambas técnicas suelen coexistir: fine-tuning para el comportamiento base y RAG para el conocimiento actualizable.

Los anti-patrones que más frecuentemente falla un proyecto RAG

1. Usar RAG como sustituto de fine-tuning para ajustar el comportamiento del modelo.
 2. Construir el índice una vez y no actualizarlo.
 3. Chunking descuidado que rompe la coherencia de los fragmentos.
 4. No implementar control de acceso a nivel de fragmento.
 5. No medir la calidad del retrieval independientemente de la calidad de la respuesta final.
-

"Un arquitecto no memoriza respuestas. Comprende problemas para poder diseñar soluciones."

Módulo 3 — Context Engineering

Capítulo 06 — RAG (Retrieval-Augmented Generation) como componente del Context Engineering

Sección 14 — Autoevaluación

"La autoevaluación no es un examen. Es la herramienta para verificar si los conceptos se comprenden o solo se recuerdan."

Instrucciones

Las preguntas siguientes están organizadas en tres niveles. Las preguntas de nivel básico verifican la comprensión de los conceptos centrales. Las de nivel intermedio evalúan la capacidad de analizar situaciones concretas. Las de nivel avanzado demandan síntesis y criterio de diseño.

Para cada pregunta de opción múltiple, identificar la respuesta correcta y justificarla. Para las preguntas abiertas, formular una respuesta de no más de cinco oraciones.

Nivel básico: comprensión conceptual

1. ¿Cuál de las siguientes afirmaciones describe mejor la relación entre RAG y el Context Engineering?
 - a) RAG es una técnica de entrenamiento que ajusta los parámetros del modelo para incluir conocimiento externo.
 - b) RAG es una estrategia del Context Engineering que incorpora conocimiento externo al contexto de cada inferencia.

- c) RAG reemplaza a las instrucciones del sistema cuando el corpus es suficientemente grande.
 - d) RAG y el fine-tuning son enfoques equivalentes que producen los mismos resultados.
-

2. ¿Cuál es la función principal de los embeddings en un pipeline RAG?

- a) Almacenar los documentos originales en un formato comprimido para ahorrar espacio.
 - b) Convertir texto en vectores numéricos donde textos con significados similares producen vectores cercanos.
 - c) Generar resúmenes automáticos de los fragmentos antes de indexarlos.
 - d) Comprimir los fragmentos para que quepan dentro de la ventana de contexto del modelo.
-

3. ¿Por qué el chunking es considerado la decisión más crítica de la fase de indexación?

- a) Porque determina el costo computacional del embedding.
 - b) Porque la estrategia de segmentación define la granularidad y coherencia de los fragmentos que el retrieval puede recuperar.
 - c) Porque el tamaño del chunk determina la latencia del modelo de lenguaje.
 - d) Porque el chunking afecta únicamente a los documentos más largos del corpus.
-

4. ¿Qué problema resuelve la búsqueda híbrida (dense + sparse)?

- a) Permite indexar documentos de cualquier idioma sin necesidad de un modelo multilingüe.
 - b) Elimina la necesidad de re-ranking al combinar dos estrategias de recuperación.
 - c) Mejora el recall para consultas que requieren tanto coincidencia semántica como coincidencia exacta de términos específicos.
 - d) Reduce el tamaño del índice vectorial combinando dos sistemas de almacenamiento.
-

Nivel intermedio: análisis de situaciones

- 5.** Un equipo desarrolla un asistente de soporte técnico que responde preguntas sobre la documentación de un producto de software. La documentación se actualiza con cada versión

del producto, que se lanza cada seis semanas. El equipo considera usar fine-tuning para especializar el modelo en el vocabulario técnico del producto.

Analizar esta situación: ¿cuándo sería preferible RAG y cuándo fine-tuning? ¿Podría justificarse el uso de ambos en conjunto?

6. Un sistema RAG implementado hace seis meses comenzó a recibir reportes de usuarios que afirman que el sistema cita información desactualizada, específicamente referenciando versiones de procedimientos que fueron revisadas en los últimos tres meses. El corpus fuente sí fue actualizado: los documentos nuevos están disponibles en el repositorio.

¿Cuál es la causa más probable del problema? ¿Qué cambio en el diseño del pipeline habría prevenido esta situación?

7. Un AI Engineer propone mejorar la calidad del retrieval agregando un modelo de cross-encoder para re-ranking. El equipo de producto objeta que la latencia del sistema ya está cerca del límite aceptable (2 segundos de extremo a extremo) y que agregar el cross-encoder aumentaría la latencia en 300-400ms adicionales.

¿Cómo evaluarías esta situación? ¿Qué alternativas podrían considerarse para mejorar la calidad del retrieval sin aumentar la latencia de forma significativa?

8. El corpus de un sistema RAG tiene 60.000 fragmentos provenientes de 3.000 documentos, de los cuales el 40% son versiones antiguas de documentos que ya tienen una versión actualizada. Ambas versiones coexisten en el índice.

¿Cómo afecta esta situación a la calidad del retrieval? ¿Qué proceso de gestión del índice debería implementarse para resolver y prevenir este problema?

Nivel avanzado: criterio de diseño

9. Diseñar en términos conceptuales un pipeline RAG para el siguiente escenario: una plataforma de e-learning con 500.000 estudiantes que necesita un asistente que responda

preguntas sobre el material de sus cursos. Cada estudiante accede solo al material de los cursos en los que está inscripto. El catálogo de cursos tiene 2.000 cursos, cada uno con materiales de entre 10 y 200 páginas. El material de cada curso se actualiza aproximadamente una vez por semestre.

Describir: estrategia de chunking, metadatos clave, cómo se implementa el control de acceso, estrategia de retrieval y política de actualización.

10. En el capítulo se presentaron cinco anti-patrones de RAG. Elegir el anti-patrón que consideres más difícil de detectar en la práctica y explicar: por qué es difícil de detectar, qué señales indirectas podrían alertar al equipo sobre su presencia, y qué métricas o procesos permitirían confirmarlo.

Respuestas de referencia (preguntas de opción múltiple)

Pregunta 1: La respuesta correcta es **b**. RAG es una estrategia del Context Engineering que enriquece el contexto enviado al modelo en cada inferencia; no modifica los parámetros del modelo ni reemplaza a las instrucciones del sistema.

Pregunta 2: La respuesta correcta es **b**. Los embeddings representan el significado del texto como posición en un espacio vectorial; la operación de retrieval busca posiciones cercanas en ese espacio.

Pregunta 3: La respuesta correcta es **b**. Si los fragmentos no son coherentes —rompen definiciones, separan condiciones de sus antecedentes, mezclan ideas de secciones distintas—, el retrieval recuperará fragmentos que el modelo no podrá usar eficientemente aunque la búsqueda vectorial sea técnicamente correcta.

Pregunta 4: La respuesta correcta es **c**. BM25 captura coincidencia exacta de términos (nombres propios, identificadores, siglas) que la búsqueda vectorial puede no recuperar si el embedding no representa esos términos con suficiente discriminación. La combinación mejora el recall global.

"Un arquitecto no memoriza respuestas. Comprende problemas para poder diseñar soluciones."

Módulo 3 — Context Engineering

Capítulo 06 — RAG (Retrieval-Augmented Generation) como componente del Context Engineering

Sección 15 — Transición al Capítulo 07

Lo que RAG resolvió y lo que no

Este capítulo construyó una capacidad específica para los sistemas de IA: acceder a conocimiento externo que el modelo no tiene en sus parámetros, recuperarlo en función de la relevancia para cada consulta e incorporarlo al contexto de la inferencia. Es una capacidad poderosa, pero tiene un alcance bien definido.

RAG resuelve bien el problema del conocimiento **semiestructurado o no estructurado que es relativamente estable**: documentos normativos, manuales técnicos, contratos, informes, bases de conocimiento institucional. El corpus puede actualizarse, pero no en tiempo real. La información existe en texto, está organizada en fragmentos, puede indexarse y puede recuperarse por similitud semántica.

Lo que RAG no resuelve es el problema del conocimiento **dinámico y transaccional**: el precio de una acción en este momento, el estado actual de un pedido, el resultado de ejecutar una consulta SQL sobre datos de producción, el contenido de una página web que cambió hace diez minutos, o el resultado de llamar a una API externa que no existía cuando se construyó el índice.

Para esa clase de información, el mecanismo de recuperación adecuado no es un índice vectorial pre-construido. Es una herramienta que el sistema puede invocar en tiempo real.

El capítulo que viene: herramientas

El Capítulo 07 introduce la capacidad de uso de herramientas (tool use o function calling): la posibilidad de que el sistema de IA invoque funciones definidas por el desarrollador para acceder a información o ejecutar acciones en tiempo real.

Donde RAG recupera fragmentos de texto de un índice pre-construido, las herramientas ejecutan operaciones: consultan una base de datos, llaman a una API, leen un archivo del sistema, envían un correo, ejecutan código.

La distinción es clara:

Característica	RAG	Herramientas (Tool Use)
Tipo de información	Semiestructurada, no estructurada	Dinámica, transaccional, en tiempo real
Mecanismo	Búsqueda por similitud en índice pre-construido	Invocación de función en tiempo de ejecución
Actualización	Periódica (minutos a días)	Instantánea (la herramienta devuelve el estado actual)
Trazabilidad	Fragmento con metadatos de fuente	Resultado de la invocación con parámetros registrados
Adecuado para	Bases de conocimiento, documentación, políticas	Precios, estado de sistemas, datos transaccionales

En la práctica, muchos sistemas de producción usan ambos mecanismos en la misma aplicación. El asistente de una empresa de logística puede usar RAG para responder preguntas sobre políticas de envío (conocimiento documental estable) y herramientas para consultar el estado actual de un envío específico (dato transaccional en tiempo real). No son competidores: son complementos con ámbitos de responsabilidad distintos.

La distinción que el módulo fue construyendo

Esta distinción entre RAG y herramientas no es nueva en el módulo. El Capítulo 02 la introdujo al comparar las estrategias para superar los límites de la ventana de contexto: la

recuperación externa (antecedente de RAG) y las herramientas aparecieron como dos formas de acceder a información que el sistema no tiene en el contexto inmediato.

El Capítulo 06 desarrolló en profundidad la primera. El Capítulo 07 desarrollará en profundidad la segunda.

Comprender ambas —y cuándo usar cada una— es una de las capacidades fundamentales del AI Engineer en el diseño de sistemas de contexto.

Lo que se viene

El Capítulo 07 cubrirá:

- Qué es el function calling y cómo el modelo decide cuándo invocar una herramienta.
 - Cómo se define y documenta una herramienta para que el modelo la use correctamente.
 - Patrones de diseño para herramientas robustas: idempotencia, manejo de errores, timeouts.
 - Herramientas de lectura versus herramientas de escritura y los distintos riesgos que presentan.
 - Cómo combinar RAG y herramientas en la misma arquitectura de sistema.
 - Anti-patrones: herramientas demasiado genéricas, herramientas sin validación de parámetros, herramientas que producen efectos secundarios no auditados.
-

Una nota sobre la progresión del módulo

Al llegar al final del Capítulo 06, el módulo ha cubierto las cuatro estrategias fundamentales del Context Engineering:

1. **Instrucciones del sistema** (Capítulo 05): definición de comportamiento, rol y restricciones.
2. **Memoria conversacional** (Capítulo 04): continuidad entre turnos y sesiones.
3. **RAG** (Capítulo 06): conocimiento externo semiestructurado, recuperado por relevancia.
4. **Herramientas** (Capítulo 07): información dinámica y acciones en tiempo real.

Los capítulos que siguen integrarán estas estrategias en arquitecturas de mayor complejidad: agentes, pipelines multi-step, sistemas de evaluación y operaciones de IA a escala.

"Un arquitecto no memoriza respuestas. Comprende problemas para poder diseñar soluciones."